



MASTER IN HIGH PERFORMANCE COMPUTING

ACCESS-OM2 Earth System Model Compatibility Between Two Supercomputers

Supervisor(s):

DR. RICCARDO FARNETI (ICTP)

DR. IRINA DAVYDENKOVA (MHPC)

Candidate:

Natalia TILININA

10th EDITION
2023–2024

ABSTRACT

The evolution of weather and climate modeling has been linked to advances in supercomputing, from the first atmospheric simulations on ENIAC in the 1950s to today’s exascale systems like El Capitan (1.7 exaflops). As Earth System Models (ESMs) have grown in complexity, integrating high-resolution atmosphere, ocean, ice, and biogeochemical components, they face critical challenges in scalability, energy efficiency, and portability across heterogeneous architectures. This study addresses these challenges through the porting of the ACCESS-OM2 ESM from Gadi (PBS, Intel Cascade Lake) to Leonardo (SLURM, Intel Sapphire Rapids) supercomputer, with a focus on workflow adaptation, cross-platform reproducibility, benchmarks and profiling. Scheduler adaptation included refactoring of the PBS and ACCESS OM2 Payu workflow manager for SLURM scheduler. ACCESS-OM2 profiling with “mpiP” and “perf” tools demonstrated the coupling between model components and I/O are the dominant bottlenecks and consume the most of the wall-clock model runtime. ACCESS-OM2 achieve weak scaling on Leonardo mostly due to not optimized NUMA usage in the ported configuration and requires further work to achieve high scalability. The study also contextualizes these results within broader ESM optimization efforts, drawing on case studies like CESM-HR on Sunway TaihuLight ($3.4\times$ speedup via cache-aware redesign) and EC-Earth3P-HR (46% runtime reduction through coupling synchronization fixes). The study also emphasizes the need for portable, scheduler-agnostic workflow managers and filesystems to match compute advancements that would support the global climate collaboration efforts, such as CMIP (Coupled Model Intercomparison Project).

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to the Scuola Internazionale Superiore di Studi Avanzati (SISSA) and the Abdus Salam International Centre for Theoretical Physics (ICTP) for organizing the *Master in High Performance Computing* program — a course that truly lived up to the phrase "rocket science" and which has significantly contributed to my academic and professional development.

Appreciation is due to course coordinator Dr. Irina Davydenkova for her excellent coordination throughout the program, and to Dr. Ivan Girotto for his contributions in all aspects. I am also grateful to all the professors and lecturers of the program for their expertise and dedication.

My deepest gratitude goes to my research supervisor, Dr. Riccardo Farneti at ICTP, for entrusting me with a challenging and stimulating scientific task for this thesis. His guidance and high standards have been instrumental in shaping this work.

I am thankful to the National Institute of Oceanography and Applied Geophysics (Istituto Nazionale di Oceanografia e di Geofisica Sperimentale, OGS) fellowship for covering the tuition costs of the course, and to the TeRABIT project for supporting the research work carried out during this period.

Profound gratitude is expressed to Dr. Paolo Lazzari of OGS for his expert review of this work. The time and expertise he devoted to this task are sincerely appreciated.

Gratitude is also extended to Harshula Jayasuriya, Aidan Heerdegen, Anton Steketee, John Reilly, Angus Gibson, and Andrew Kiss from the Australian Community Climate and Earth System Simulator National Research Infrastructure (ACCESS-NRI) and the Consortium for Ocean–Sea Ice Modelling in Australia (COSIMA) for their patient and detailed responses to my numerous posts and inquiries on the ACCESS-Hive forum.

I am sincerely grateful to Prof. Sergey Gulev of the Shirshov Institute of Oceanology for his continuous and invaluable support throughout my scientific journey.

Contents

Abstract	1
Acknowledgements	2
1 Introduction	5
1.1 Historical Development of Weather Simulations and Supercomputing . .	5
1.2 Scalability of Earth System Models in the Exascale Computing Era . .	8
1.2.1 Case Study 1: CESM-HR on Sunway TaihuLight	8
1.2.2 Case Study 2: ECMWF IFS CY43R3 – I/O Optimization . . .	10
1.2.3 Case Study 3: EC-Earth3P-HR Performance Analysis	10
1.2.4 Case Study 4: NEMO Ocean Model on Sunway Supercomputer	14
1.2.5 Case Study 5: ICON-MSA on JUWELS HPC System	16
1.3 Challenges of Quantum Computing for Climate Modeling	18
1.4 Motivation	20
2 Leonardo and Gadi Supercomputers Architecture	23
2.1 PBS and SLURM schedulers	24
3 Porting the ACCESS-OM2 Earth System Model from Gadi Supercomputer to Leonardo Supercomputer	26
3.1 ACCESS-OM2 Earth System Model	26
3.2 Technical Migration to Leonardo: Compilation and Setup	30
3.2.1 Initial Conditions and Repeat Year Forcing	31
3.2.2 Model Compilation and Platform Adaptation	33
3.2.3 Compilation Using the ACCESS-NRI Spack-Based Strategy . .	37
3.3 Adjustment of the Payu Workflow Manager for SLURM Job Submission	43
3.4 Running Model Experiments and Postprocessing Simulation Output . .	45

4	Performance Profiling of ACCESS-OM2 on the Leonardo Supercomputer with perf and mpiP tools	50
4.1	Runtime analysis of MOM5 and CICE5	51
4.2	Performance Analysis with mpiP: Lightweight MPI Profiling	54
4.3	Profiling with perf tool	60
5	Summary and Perspectives	69
5.1	Summary	69
5.2	Perspectives	70
	References	75

1 Introduction

1.1 Historical Development of Weather Simulations and Supercomputing

The development of numerical weather simulations and supercomputers has been deeply intertwined. The first electronic computer, ENIAC, ran an atmospheric model in the 1950s with a resolution of 500–800 km. In the 1960s, Seymour Cray’s CDC 6600 (Control Data Corporation), an early supercomputer, was installed at NCAR (National Center for Atmospheric Research) to advance atmospheric studies. Over the decades, supercomputing performance has surged from 1 Mflops (10^6 floating-point operations per second, e.g., CDC 3300) to 1.7 exaflops (10^{18} flops) for El Capitan supercomputer at Lawrence Livermore National Laboratory (LLNL), which topped the November 2024 TOP500 supercomputers list.

This leap was enabled by several factors: the evolution of semiconductor technology (the regular increase in frequency stopped more than a decade ago; since then, computing power has largely increased due to a higher density of computing units in processors), architectural shifts toward power-efficient heterogeneous many-core systems, breakthroughs in GPU acceleration, and advancements in interconnects (e.g., InfiniBand), which enable low-latency, high-bandwidth data transfer within supercomputing networks. Together, these innovations have made modern exascale systems capable of running Earth system models (ESMs) at unprecedented spatial and temporal resolutions.

By the early 2010s, fundamental physics limitations had halted the decades-long trend of processor frequency increases that had previously driven Moore’s Law. This technological inflection point forced a radical rethinking of computational approaches, shifting the focus to parallel processing density. Consequently, post-2010 supercomputer designs increasingly incorporated many-core accelerators as primary compute engines, for example, NVIDIA GPUs ([Vazhkudai et al., 2018](#)).

This transition introduced two transformative architectural changes to high-performance computing systems. First, homogeneous architectures gave way to heterogeneous systems combining multiple processor types - a shift that necessitated fundamentally new programming models and scheduling approaches to effectively utilize diverse computational resources. Second, core counts increased exponentially while individual core complexity decreased, creating systems with hundreds of thousands of simplified processing elements.

These architectural innovations, combined with advancements in high-bandwidth interconnects (e.g., InfiniBand) and memory hierarchies, presented both opportunities and challenges. While enabling unprecedented computational throughput, they required complete re-engineering of traditional algorithms and data structures to achieve efficient parallel execution. The successful adaptation to this new paradigm has been crucial for enabling modern exascale systems to run Earth System Models (ESMs) at previously unattainable spatial and temporal resolutions, representing a quantum leap in climate modeling capability.

In parallel, Numerical Weather Prediction (NWP) models have evolved into complex Earth system models (ESMs), integrating once-isolated components – such as atmospheric dynamics, land-surface interactions, ice sheets, and biogeochemical cycles – into unified frameworks. Today’s ESMs resolve small-scale processes like cloud microphysics and turbulence in the ocean and atmosphere at very high resolutions and quantify uncertainties through ensemble forecasting.

However, as ESMs grew more sophisticated, they often incorporated increasingly complex physics, parameterizations, and thousands of lines of code, with a limited focus on optimizing them for modern supercomputer architectures. This was in part because ESM developers were traditionally scientists who specialized in meteorology, oceanography, climate science, and mathematics rather than high-performance computing (HPC). Moreover, ESMs are usually housed in research groups for several decades and are being developed by several generations of scientists. Most of these developments only focus on a sub-problem and contribute to a subroutine or external

module of the entire model. As a result, the modern ESM consists of hundreds of modules that were written at different times and even with different versions of programming languages. The complexity in both the code and the science is a challenge for moving the code to different and more modern architectures (for example, using GPU accelerators), or to port the ESM from one supercomputer to another. Facing the problem of tens of thousands of lines of code, the researchers and developers have to either perform an extensive rewriting of the code [Xu et al. \(2015\)](#) or invest years of effort into redesign methodologies and tools [Gysi et al. \(2015\)](#).

The second major challenge comes from the fact that unlike many HPC applications dominated by a few computational "hot spots", ESMs exhibit a flat profile with hundreds of moderately important kernels, each representing different component of the climate system. This distributed pattern means performance optimization requires examining and potentially modifying numerous modules - a substantially more complex than optimizing the "hot spot" kernels.

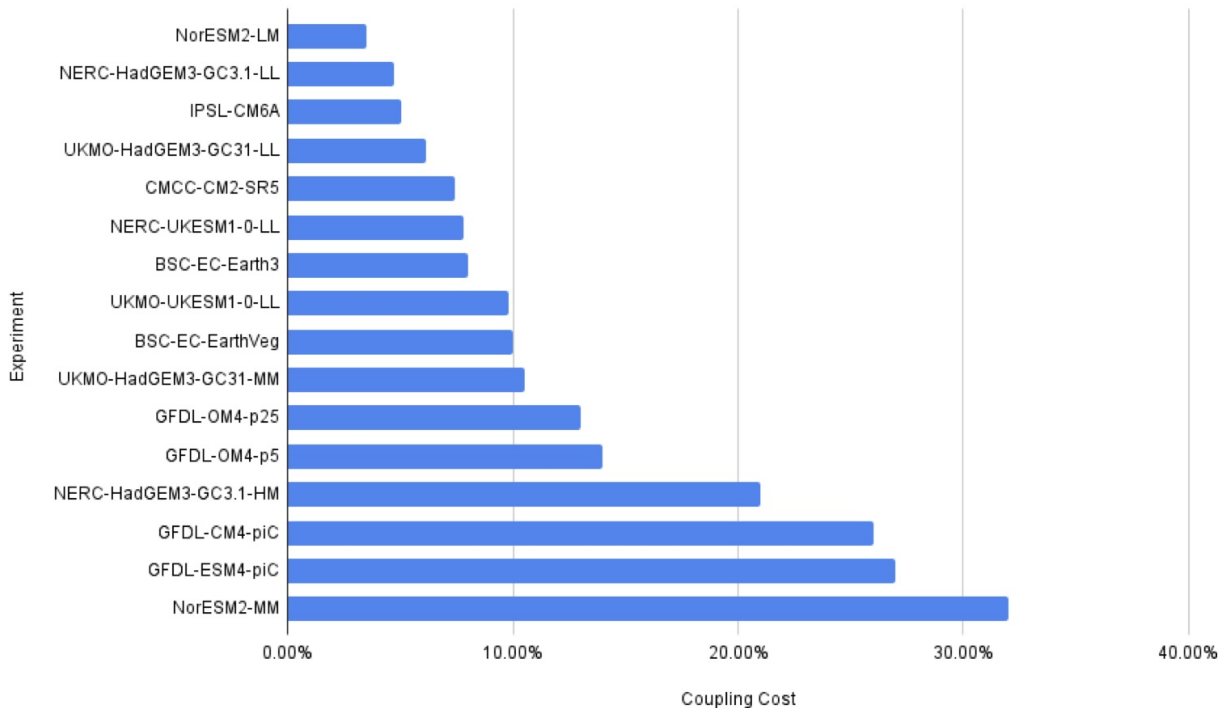


Figure 1.1: Coupling cost of CMIP6 modules for all the institutions that provided the metric. Adapted from [Acosta et al. \(2024\)](#), original Fig. 4.

Another, more ethical dilemma is the energy efficiency of ESMs [Acosta et al. \(2024\)](#).

This paradox is most apparent in the Coupled Model Intercomparison Project (CMIP), which while providing the scientific foundation for greenhouse gas mitigation strategies through IPCC reports, simultaneously generates substantial carbon emissions through its computational demands. [Acosta et al. \(2024\)](#) estimated carbon footprint of running CMIP6 simulations as 1692 tons of CO₂ equivalent, even when accounting for the experiments executed by only 8 out of the 49 institutions that are enlisted in CMIP6. The study showed that between 5% – 15% of the execution cost is spent in the coupling between independent components (Fig. 1.1, original Fig. 4 in [Acosta et al. \(2024\)](#)), and it only gets worse by increasing the number of resources. Therefore, reducing the energy consumption is the another major factor that drives refactoring and optimization of ESMs to new architectures.

1.2 Scalability of Earth System Models in the Exascale Computing Era

Over the last decade, a growing body of research (e.g., [Du et al., 2025](#); [Rasch et al., 2019](#); [Haarsma et al., 2020](#); [Adams et al., 2019](#); [Yepes-Arbós et al., 2022](#); [Bishnoi et al., 2024](#); [Zhang et al., 2020](#)) has emphasized the need to optimize ESMs for contemporary supercomputers, addressing bottlenecks in scalability, memory usage, and parallelization. While not comprehensive, the optimization examples presented in this section demonstrate representative strategies used in Earth System Model (ESM) development.

1.2.1 Case Study 1: CESM-HR on Sunway TaihuLight

The comprehensive study of [Zhang et al. \(2020\)](#) illustrates the transformative potential of code adaptation for next-generation supercomputers. In their study the high-resolution Community Earth System Model (CESM-HR, 25-km atmosphere/10-km ocean) was ported from Intel’s homogeneous x86 architecture to China’s Sunway’s Management Processing Element (MPE)-only TaihuLight system, a pioneering heterogeneous many-core architecture.

Their analysis revealed a critical bottleneck: CESM-HR’s performance was constrained by memory bandwidth rather than raw processing power - the model wasn’t limited by the computer’s processing power, but rather by how quickly data could move through the system (memory bandwidth). Through refactoring and parallelization of legacy code, they achieved a $3.4\times$ acceleration in simulation speed – increase from 1 to 3.4 simulation years per day (SYPD). Key adjustments allowed for acceleration:

- **Cache-aware algorithm redesign:** Restructured computational kernels to maximize data locality and minimize cache misses, particularly for the spectral element dynamical core. NUMA-aware data placement across Sunway’s 4 memory controllers.
- **Data structure optimization:** Reorganized memory layout to enable better utilization of the Sunway’s 32KB L1 cache and 256-bit vector units.
- **Fine-grained parallelization:** Implemented across 10,649,600 processor cores (40,960 nodes) with dynamic load balancing for heterogeneous workloads.
- **Thread-level parallelism:** Optimized OpenMP directives to exploit all 260 cores per SW26010 processor. Replaced OpenMP’s dynamic scheduling with static block distribution.
- **MPI/OpenMP hybrid parallelization:** Combined coarse-grained domain decomposition (MPI) with fine-grained loop-level parallelism (OpenMP), reducing communication overhead.
- **Memory access pattern restructuring:** Implemented software prefetching to overcome memory bandwidth limitations.

This study (among others) established a template for adaptation of legacy Fortran code to next generation of supercomputer architectures like GPUs and neuromorphic chips, while highlighting the growing importance of memory hierarchy optimization in exascale climate modeling.

1.2.2 Case Study 2: ECMWF IFS CY43R3 – I/O Optimization

The study by [Yepes-Arbós et al. \(2022\)](#) examines the critical importance of efficient input/output (I/O) strategies in high-resolution climate modeling, addressing systemic bottlenecks in the European Centre for Medium-Range Weather Forecasts (ECMWF) operational Integrated Forecasting System (IFS CY43R3). Their work reveals that legacy sequential I/O schemes—forcing serialized data writing across thousands of parallel processes—introduce significant performance penalties as model resolution increases (Fig. 1.2, original Fig. 2 in [Yepes-Arbós et al. \(2022\)](#)).

As shown in Figure 1.2, the sequential output scheme produces significant serialization during data gathering operations. Remarkably, the time required for gathering, encoding, and writing output equals the computational cost of three entire model time steps. This substantial overhead highlights the critical need for optimized I/O approach.

The study presents an effective solution through parallel I/O server implementation, which enables efficient model execution. This improved performance is achieved through two key innovations:

- Asynchronous communication patterns using non-blocking MPI operations (particularly `MPI_Isend/MPI_Irecv`) to overlap computation and I/O.
- Decoupled I/O processes with dedicated I/O server ranks that handle file operations while compute ranks continue processing.
- Optimized data distribution across I/O servers to prevent new bottlenecks.

1.2.3 Case Study 3: EC-Earth3P-HR Performance Analysis

[Haarsma et al. \(2020\)](#) conducted a comprehensive performance analysis of EC-Earth3P (~ 80 km atmosphere, 1.0° ocean) and its high-resolution variant EC-Earth3P-HR (~ 40 km atmosphere, 0.25° ocean) for the HighResMIP/CMIP6 project. The study revealed critical scalability limitations in the coupled system. While individual model

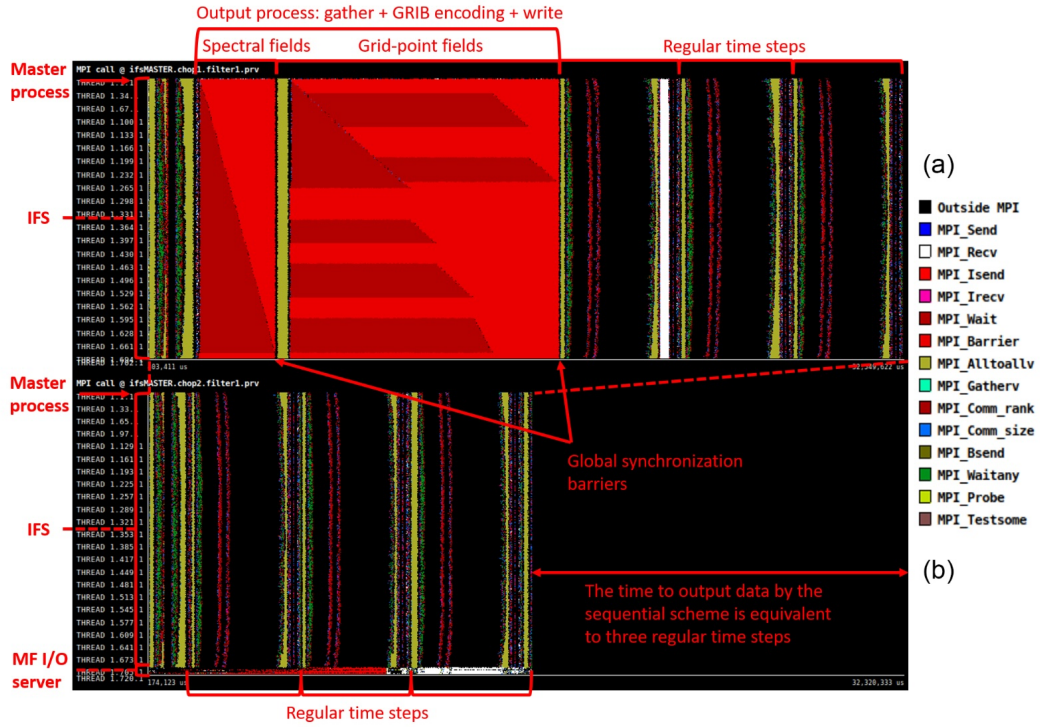


Figure 1.2: Panel (a) illustrates the serialisation caused by the sequential output scheme, whereas panel (b) illustrates the fact that IFS with the internal parallel I/O server runs efficiently with no observable delays. Both traces have the timeline on the x axis and the MPI processes on the y axis. Along the timeline, many MPI call events happen, with each colour representing an MPI function. Adapted from Yepes-Arbós et al. (2022), original Fig. 2.

components (IFS atmospheric model and NEMO ocean model) showed good scaling from 1 to 6 simulated years per day (SYPD) when number of processors increase from 240 to 1920, the fully coupled system using the OASIS coupler only achieved 1 to 3 SYPD with an 8-fold processor increase.

The analysis identified three major bottlenecks through PARAVAR parallel performance traces (Fig. 1.3, original Fig. 2 in [Haarsma et al. \(2020\)](#)). User could experience that the speeding up of one component (e.g., the reduction of the execution time of IFS) does not reduce the execution time of the coupled application. The main bottleneck for this is one synchronization point at the end each coupled time step where both components (NEMO and IFS) exchange fields. The coupling synchronization point consumed over 50% of runtime at 1024 processes, with IFS-to-NEMO field exchanges occurring in four separate communication groups. Second, NEMO exhibited inefficient global communications (shown in purple in traces) particularly in its LIM3 ice model routines. Third, the sequential I/O implementation forced 30% runtime overhead as a master process gathered output data while other processes waited.

Several optimizations yielded significant improvements:

- OASIS3-MCT’s “opt” option reduced coupling time by 90% by replacing all-to-one communications with gather/scatter operations.
- Field gathering optimization consolidated communications from four to two groups, cutting coupling overhead by 40%.
- XIOS parallel I/O server implementation eliminated the sequential output bottleneck.
- ELPiN domain decomposition tool improved NEMO efficiency by 41% while using 25% fewer resources.

These optimizations collectively reduced total execution time by 46% for EC-Earth3P-HR. The study highlights that while high-resolution climate modeling presents substantial computational challenges, targeted optimizations of coupling,

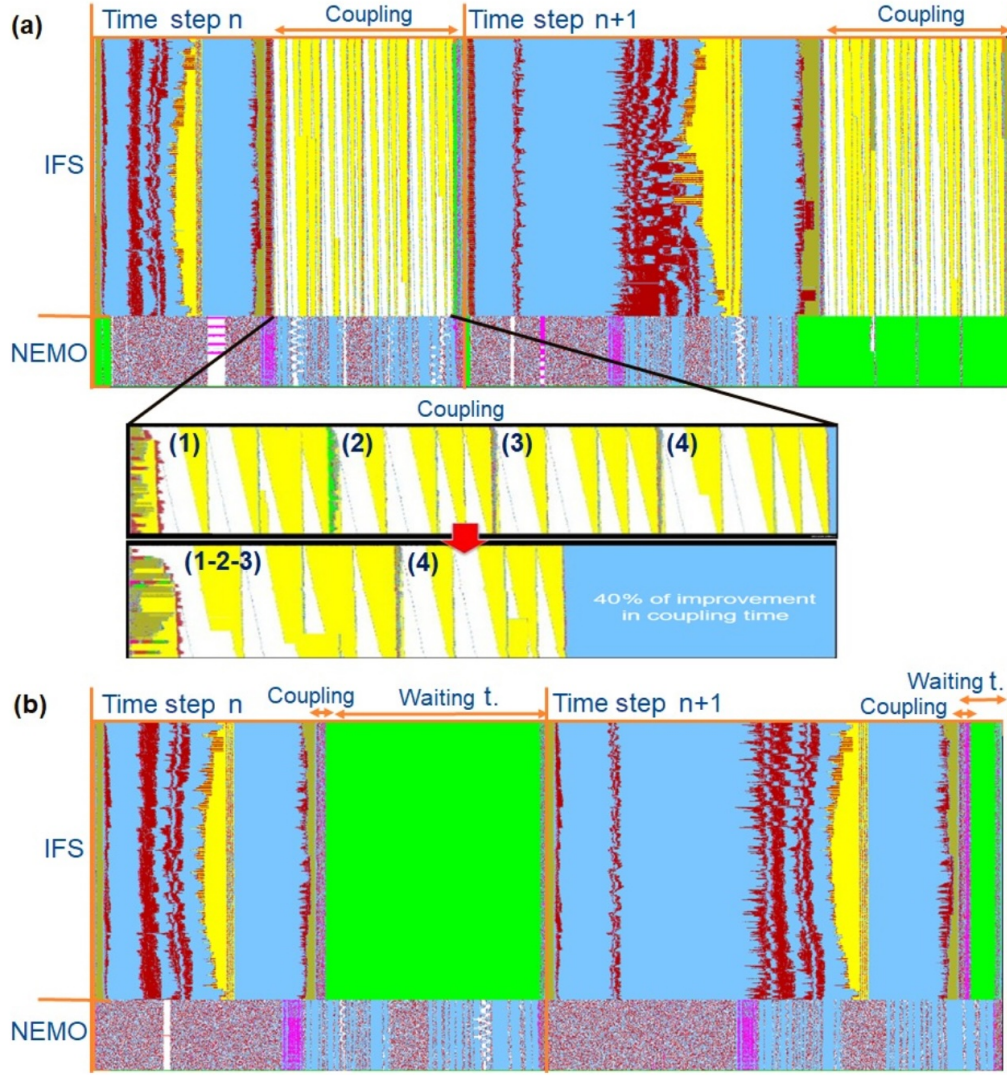


Figure 1.3: (a) PARAVIEW view of the NEMO and IFS components in an EC-Earth3P-HR model execution for two time steps including the coupling process. The horizontal lines give the behavior of the different processes (1 to 512 for IFS and 513 to 536 for NEMO) as a function of time. Each color corresponds to a different MPI communication function. See text for explanation. Panel (b) is the same as (a) but when optimization options “opt” and “gathering” for coupling are activated. Adapted from [Haarsma et al. \(2020\)](#), original Fig. 2.

communication, and I/O patterns can dramatically improve model performance and scalability.

Looking forward to the future, it is anticipated that HPC architectures will change radically, and that the diversity of architectures will increase. Even when porting from one CPU-based machine to another, there is a cost due to the need to re-tune the performance for the particular characteristics of the new machine. With the emergence of more diverse machines such as GPU-based machines, the cost will only get worse.

1.2.4 Case Study 4: NEMO Ocean Model on Sunway Supercomputer

Another good example of adaptation of the OGCM (Ocean General Circulation Models) to modern computing architectures is the study of [Ye et al. \(2022\)](#). The authors guide through the development of swNEMO_v4.0 on the new Sunway supercomputer and demonstrate significant overcoming of scalability limitations in ocean general circulation models (OGCMs). The implementation demonstrates remarkable performance at extreme scale, achieving unprecedented parallel efficiency of 99.29% when running at 500 m resolution using 27,988,480 cores (Fig. 1.4, original Fig. 12 in [Ye et al. \(2022\)](#)).

Benchmarking with GYRE-PISCES demonstrated:

- 1.97 PFLOPS peak performance with 27,988,480 cores.
- $10.25\times$ speedup over the original version.
- $7.53\times$ improvement in simulated years per day (SYPD).
- Parallel efficiency of 71.48%, 83.40%, and 99.29% for resolutions of 2 km, 1 km, and 500 m, respectively.

It is worth to mentioning that the Sunway TaihuLight supercomputer utilizes domestically developed in China semiconductors, featuring 40,960 SW26010 manycore processors based on the Sunway architecture. Each processor chip contains 256

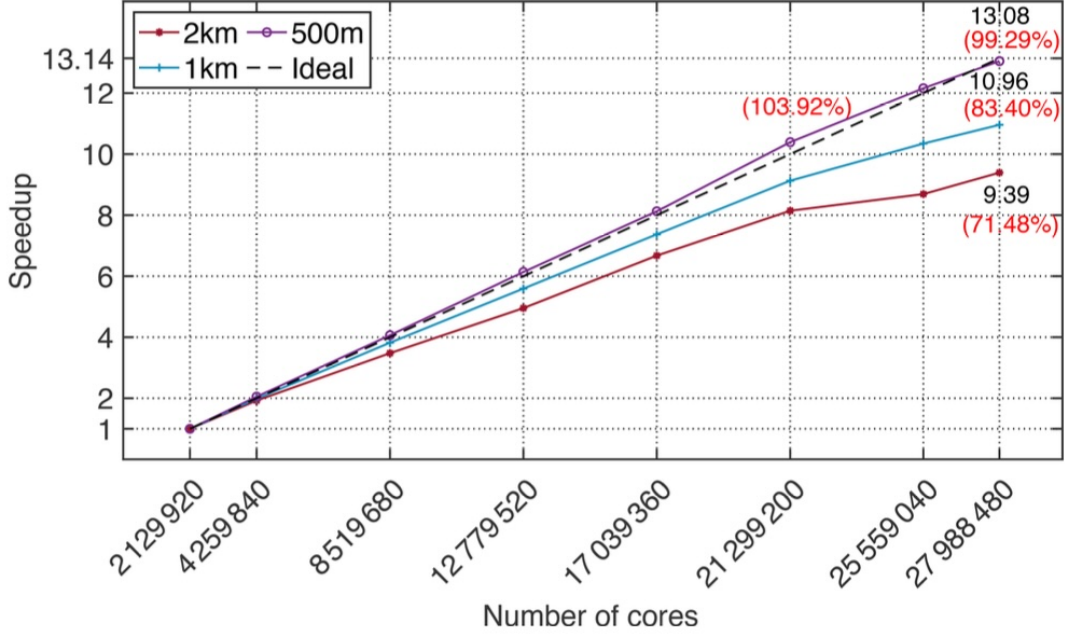


Figure 1.4: The strong scalability results, scaling from 2 129 920 to 27 988 480 cores with 2 km, 1 km and 500 m resolutions. Adapted from [Ye et al. \(2022\)](#), original Fig. 12.

processing cores and four auxiliary cores for system management. These cores are equipped with 64 KB of scratchpad memory for data and 16 KB for instructions, with communication facilitated via a network on chip, bypassing a traditional cache hierarchy.

This breakthrough was enabled by four key innovations:

- **Load-Balanced Longitude–Latitude Decomposition:** The simulation domain is decomposed along longitude and latitude axes, ensuring even workload distribution across MPI processes and minimizing communication overhead.
- **Asynchronous MPE–CPE Parallelization:** Efficient data exchange between the Master Processing Element (MPE) and CPE cluster is achieved using DMA (Direct Memory Access), enabling concurrent computation and communication.
- **Latitude–Depth Decomposition within CPE Clusters:** RMA (Remote Memory Access) is used for fine-grained parallelism and optimized on-chip communication to enhance depth-wise computation.

- **Vectorization and Data Layout Optimization:** Memory structures are redesigned to leverage the 512-bit SIMD capabilities of the SW26010, optimizing vectorized computations.

Stencil-based calculations, essential to NEMO, are optimized using RMA to improve data exchange, minimizing memory bandwidth bottlenecks. A novel cache management strategy dynamically balances Local Data Memory (LDM) and LDCache, achieving $88\times$ speedup over traditional methods by maximizing memory bandwidth utilization. A hybrid approach utilizing half, single, and double precision reduces memory footprint while maintaining accuracy, achieving nearly $2\times$ speedup over double precision with minimal computational bias. These advancements enable high-resolution ocean simulations that capture small-scale processes such as sub-mesoscale eddies and internal waves, improving ocean and climate forecasting. The optimizations are applicable to other OGCMs and supercomputers.

1.2.5 Case Study 5: ICON-MSA on JUWELS HPC System

Another study that addressed the confrontation of complex ESM codes with novel supercomputing architectures is [Bishnoi et al. \(2024\)](#). Authors provided a comprehensive performance evaluation for the ICON model in its fully coupled atmosphere-ocean configuration (ICON-MSA) on the JUWELS modular HPC system. Their study represents the first demonstration of a 5 km global coupled climate model running efficiently across heterogeneous GPU-CPU architectures.

The atmosphere was computed on JUWELS Booster GPUs, with key kernels including spectral dynamics and cloud microphysics ported using OpenACC directives, while the ocean and I/O operations remained on the CPU Cluster, both operating at a 5 km resolution. An optimal configuration was achieved with 84 GPU nodes on JUWELS Booster and 80 CPU nodes on JUWELS Cluster (Fig. 1.5), 63 for ocean simulation, 17 for I/O, minimizing coupler waiting times. Compared to a CPU-only simulation, ICON-MSA reduced energy consumption by 59% while maintaining similar runtimes.

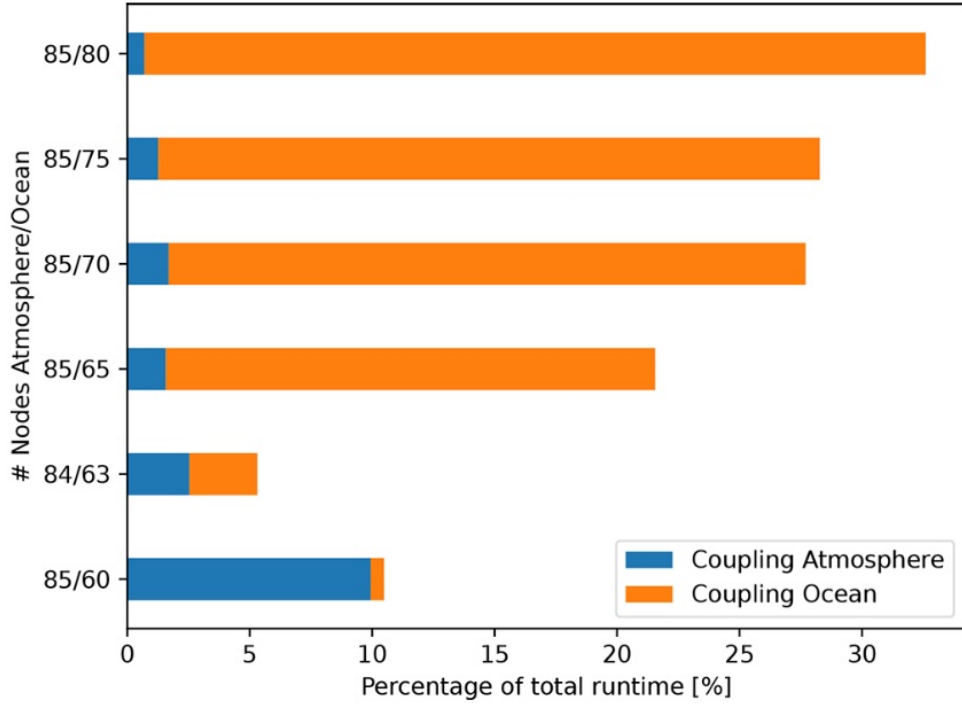


Figure 1.5: Sweet spot analysis to find a configuration of nodes that minimizes the total coupling time. On the y axis, tuples of values represent the number of JUWELS Booster and Cluster nodes used for a simulation run. The first value of each tuple signifies the number of nodes used by ICON-A for this run, and the second value of the tuple signifies the total number of nodes used by ICON-O. The 17 Cluster nodes dedicated to I/O are not taken into account since the number of I/O nodes is kept constant across all experiments. Adapted from [Bishnoi et al. \(2024\)](#), original Fig. 3.

Performance results revealed several key achievements. The configuration achieved 170 simulation days per day at maximal scaling using 335 GPU nodes and 268 CPU nodes, demonstrating 82% strong scaling efficiency. GPU utilization reached 78% of theoretical peak performance, while energy measurements showed a 59% reduction compared to CPU-only baselines. Detailed analysis identified memory bandwidth as the primary limiter for atmosphere computations on GPUs, utilizing 82% of the A100’s 1,555 GB/s bandwidth, while ocean computations remained latency-bound due to unstructured mesh operations.

Study of [Bishnoi et al. \(2024\)](#) establishes critical design patterns for exascale climate modeling, demonstrating optimal resource allocation through a 1:0.8 GPU-to-CPU node ratio and showing how energy-aware scheduling through GPU power capping can yield significant savings. The modular code design enables component-specific acceleration while maintaining flexibility for future architectural developments.

1.3 Challenges of Quantum Computing for Climate Modeling

The intersection of quantum computing and atmospheric science has garnered increasing attention due to quantum computers’ theoretical ability to outperform classical systems in solving certain computationally intensive tasks. While quantum computing is an emerging tool for many computational tasks, its application to numerical weather prediction (NWP) and climate modeling remains a fundamentally new avenue of research. To date, only the theoretical feasibility of this approach has been explored in study by [Tennie and Palmer \(2023\)](#). Their study provides the first comprehensive examination of how quantum algorithms might address the challenges of atmospheric and climate simulations, though practical implementations remain unrealized. Foundational concepts of quantum computing distinguish between classical bits and quantum bits (qubits). Unlike classical bits which hold binary states (0 or 1), qubits can exist in superpositions of both states simultaneously, allowing for an exponentially larger state space. This property, along with entanglement—correlations that exceed classical bounds—enables quantum algorithms to process complex datasets

more efficiently than classical systems under ideal circumstances (Nielsen and Chuang, 2010). One of the pivotal challenges in applying quantum computing to atmospheric modeling lies in the nonlinearity of the governing equations, such as the Navier–Stokes equations. Traditional quantum mechanics, described by the linear Schrödinger equation, is fundamentally at odds with such nonlinearity. Tennie and Palmer (2023) explore a novel workaround inspired by recent advancements (Lloyd et al., 2020), employing interaction Hamiltonians to simulate nonlinear dynamics through a mean-field approach.

In their simulation, the authors demonstrate a quantum solver applied to a basic nonlinear differential equation, showing strong concordance with classical solutions for a limited number of integration steps. Notably, they point out that mean-field errors accumulate over time, but that the quantum algorithm still offers a significant efficiency gain over earlier proposals which required exponentially increasing resources (Leyton and Osborne, 2008). While current quantum hardware is insufficient to implement these algorithms at scale, the groundwork laid by such emulations is crucial for future practical applications.

Despite these advances, Tennie and Palmer (2023) offer a realistic perspective on current quantum limitations. One of the most striking bottlenecks is the limited ability to read out information from quantum systems. While a quantum state might encode a vast amount of information, only a number of bits equivalent to the number of qubits used can be extracted at a time. This contrasts sharply with classical models, where individual variables at any grid point can be directly accessed.

This readout limitation fundamentally constrains the feasibility of using quantum systems for high-resolution weather forecasts or climate projections. While it may suffice for answering specific probabilistic questions (e.g., “Will it rain in a specific region?”), the approach is inadequate for generating full forecast fields, which are essential for operational meteorology.

"The Noisy" problem addressed in Tennie and Palmer (2023) is the inherent instability of quantum systems. Qubits are highly susceptible to decoherence due

to environmental interactions, which introduces stochastic noise. Although noise is typically viewed as detrimental, [Tennie and Palmer \(2023\)](#) suggest it might have a silver lining in the context of weather modeling. Noise-injected simulations may mimic ensemble forecasting or stochastic parametrization methods that are already widely used in classical NWP models ([Palmer, 2019](#)). This opens the door to exploring quantum algorithms capable of integrating stochastic differential equations. Early attempts to integrate lower-dimensional chaotic systems like the Lorenz attractor using quantum solvers hint at potential synergy between stochastic quantum dynamics and atmospheric modeling needs, although significant research is still needed to verify scalability and robustness.

Overall while quantum computing shows immense promise for specific tasks—such as solving large linear systems or simulating ensemble averages—it remains hampered by issues of scalability, measurement bottlenecks, and hardware limitations. Hybrid approaches, which combine classical and quantum systems, may offer a viable near-term solution. [Tennie and Palmer \(2023\)](#) study calls for interdisciplinary collaboration between quantum physicists, applied mathematicians, and atmospheric scientists. As quantum hardware evolves and quantum error correction becomes more practical, the feasibility of using these systems in operational meteorology may increase. For now, quantum computing remains a compelling but distant prospect for full-scale weather and climate prediction.

1.4 Motivation

High-performance computing (HPC) is the backbone of modern Earth system modeling, enabling the simulation of complex climate processes at high spatial and temporal resolutions. A numerous case studies presented in this chapter demonstrate endless perspective for enhancement of ESMs performance: acceleration, scalability, energy efficiency, running on fine scales and many others. While these performance enhancements are critical for advancing Earth system modeling, there is a fundamental prerequisite: *portability* across diverse HPC architectures. Current ESMs face

significant portability challenges, particularly for complex coupled configurations, due to architecture-specific dependencies in legacy Fortran, non-uniform MPI/OpenMP implementations across vendors and divergent accelerator programming models. As models such as the ACCESS-OM2 Earth System Model (Kiss et al., 2020) grow in complexity to better capture the intricacies of atmosphere-ocean-ice interactions a major challenge that emerges is ensuring model compatibility and portability across heterogeneous HPC environments, while maintaining numerical consistency and reproducibility of results.

This thesis is motivated by the necessity to evaluate the cross-platform performance and portability of the ACCESS-OM2 ESM, focusing specifically on two supercomputers: Leonardo, based at CINECA (2023) (Italy), and Gadi, hosted by Australia’s National Computational Infrastructure (NCI) (2023). Both systems represent state-of-the-art facilities, optimized for large-scale parallel computation on CPU-bound applications.

The differences in job scheduling systems (Leonardo’s SLURM versus Gadi’s PBS) and software stack configurations require significant adaptation of the ACCESS-OM2 workflow, including the modification of the Payu experiment manager and optimization of compilation strategies. These adaptations must preserve not only runtime performance but also scientific reproducibility, ensuring that simulations executed on one platform yield results that are consistent with those on another. This is especially critical for climate projections used in multi-model ensembles and intercomparison projects, such as the Coupled Model Intercomparison Project (CMIP) Eyring et al. (2016); Griffies et al. (2016).

The ACCESS-OM2 model is one of Australia’s key contributions to CMIP6 and has been extensively used in the Intergovernmental Panel on Climate Change (IPCC) Sixth Assessment Report (AR6). Given the international importance of CMIP-aligned simulations in shaping climate policy and adaptation strategies, it is imperative that ACCESS-OM2 remain portable, reproducible, and scalable across various computing systems worldwide. The ability to execute identical simulations across different

architectures not only improves scientific transparency but also facilitates global collaboration, enabling researchers across institutions and countries to compare, validate, and jointly analyze climate model outputs under harmonized experimental designs.

Moreover, with the advent of exascale computing, institutions are migrating from traditional CPU-dominated clusters to heterogeneous systems incorporating GPU acceleration and advanced memory subsystems. Ensuring that given ESM can leverage these architectures without extensive reengineering is crucial for its relevance. The work presented in this thesis contributes to this goal by porting ACCESS-OM2 from Gadi to Leonardo, adjusting the build system, managing environmental dependencies, adapting the I/O strategy, and evaluating the performance through benchmarking and profiling.

2 Leonardo and Gadi Supercomputers Architecture

The study is based on two leading supercomputing systems with distinct architectures:

Leonardo is pre-exascale and one of Europe’s most powerful high-performance systems located at CINECA, Italy’s premier HPC center. It is developed under the EuroHPC Joint Undertaking and named after Renaissance polymath Leonardo da Vinci. It features a hybrid architecture combining traditional CPU nodes with powerful GPU accelerators. Its architecture delivers exceptional performance for complex scientific workloads, particularly in climate modeling, materials science, and AI research.

Gadi serves as Australia’s flagship supercomputing facility, hosted by the National Computational Infrastructure (NCI) at the Australian National University in Canberra. Named after the local Ngunnawal language word meaning ‘to search for’, Gadi reflects Australia’s commitment to indigenous heritage while advancing cutting-edge research. As one of the most advanced HPC systems in the Southern Hemisphere Gadi supports over 5,000 researchers annually across diverse scientific domains.

Table 2.1 provides key HPC architecture specifications of Leonardo and Gadi systems relevant for this study.

	Leonardo	Gadi
Partition used	Data Centric General Purpose (112 cores/node)	48 cores/node
Processor Type	Intel Sapphire Rapids 2×56 cores	Intel Xeon Scalable Cascade Lake 2×24 cores
GPU Type	NVIDIA V100 GPUs	NVIDIA A100 GPUs
Theoretical Peak Performance (TOP 500)	306 PFlop/s (9th)	15 PFlop/s (130th)
Memory Capacity	110 PB	930 TB
Interconnect	InfiniBand Dragonfly+ 200 Gb/s	InfiniBand Dragonfly+ 200 Gb/s
Job Scheduler	SLURM	PBS

Table 2.1: Leonardo and Gadi technical specifications.

2.1 PBS and SLURM schedulers

A key architectural difference between Leonardo and Gadi relevant for porting a complex coupled Earth System Model from Gadi to Leonardo lies in their workload managers - Gadi employs PBS while Leonardo uses SLURM.

The migration of our completed climate model from PBS to SLURM represented substantial modifications to job submission scripts, resource allocation parameters, and parallel task distribution. This section details the specification differences between these two scheduling systems, with particular attention to the features that affected our model’s transition. **SLURM** (Simple Linux Utility for Resource Management) and **PBS** (Portable Batch System) are two widely used job scheduling and resource management systems used in High-Performance Computing (HPC) environments. They both manage the allocation of computational resources and job scheduling for clusters or supercomputers. However, SLURM and PBS differ in several technical aspects such as architecture, features, scalability, performance, ease of use, and community support. Table 2.2 provides comparison of PBS and SLURM specifications, focusing on the features most relevant to climate modeling workloads.

	SLURM	PBS
Description	Developed and maintained by SchedMD. Widely used in academic and commercial environments, known for its scalability, flexibility, and high performance.	Initially developed by NASA Ames Research Center. The open-source variant (OpenPBS) is available. Known for its robust feature set and used in large HPC systems.
Scalability	Known for high scalability in large clusters and supercomputing environments.	Scalable but generally considered less flexible in highly parallel environments compared to SLURM.
Hardware/Software	Supports a wide range of hardware and software environments.	Supports a wide variety of hardware but is more rigid compared to SLURM.
Installation	Easy installation and maintainability.	Requires more configuration, but also widely adopted for large systems.
Community Support	Strong community support, as it is open-source and widely used in academia.	Smaller community for OpenPBS, but PBS Pro offers commercial support.
Features	Decentralized with slurmd daemons on each node, controlled by slurmctld.	Centralized server (mom_server) with execution hosts (mom_daemons), Machine Oriented Mini-server.
Job Submission	Uses <code>sbatch</code> (e.g., <code>sbatch script.slurm</code>).	Uses <code>qsub</code> (e.g., <code>qsub script.pbs</code>).
Resource Allocation	Dynamic allocation via SLURM parameters (e.g., <code>#SBATCH -nodes=2 -ntasks-per-node=8</code>).	Static allocation by default via PBS directives (e.g., <code>#PBS -l nodes=2:ppn=8</code>).
Parallel Jobs (MPI)	Direct integration with <code>srun</code> for MPI tasks.	Requires <code>mpiexec</code> with PBS environment variables.
Climate Model Porting	Growing (most modern supercomputers, EuroHPC, NSF clusters).	Declining (legacy systems, some academic HPCs), legacy scripts require PBS-specific directives.

Table 2.2: SLURM and PBS schedulers specification.

3 Porting the ACCESS-OM2 Earth System Model from Gadi Supercomputer to Leonardo Supercomputer

3.1 ACCESS-OM2 Earth System Model

The ACCESS-OM2 is a part of the Australian Community Climate and Earth System Simulator (Kiss et al., 2020). It represents a research tool for climate model development and operational forecasting. It is open-source and freely available through the ACCESS NRI (<https://github.com/ACCESS-NRI/ACCESS-OM2>, last access: 17 April 2025). The model forms the ocean–sea ice component of Australia’s national climate modeling efforts and contributes to the development of models for the Coupled Model Intercomparison Project (CMIP6, ongoing CMIP7, and future phases). It has been evaluated with other state-of-the-art ocean–sea ice models participating in the CMIP experiments (over 100 climate model formulations developed by 49 different modeling groups worldwide) and demonstrated robust and consistent performance.

It provides configurations at three distinct horizontal resolutions: coarse (1°), eddy-permitting (0.25°), and eddy-resolving (0.1°), allowing it to be used across a range of applications from process studies to fully coupled climate simulations.

ACCESS-OM2 is driven by a prescribed atmosphere and enables a relatively clean assessment of the ocean component’s resolution dependence. However, as it lacks feedback mechanisms from sea surface temperature and currents onto the atmosphere, the model is more strongly driven by atmospheric forcing than a fully coupled ocean–atmosphere model.

ACCESS-OM2 (Fig. 3.1) integrates the Modular Ocean Model (**MOM5**) developed by the Geophysical Fluid Dynamics Laboratory (<https://mom-ocean.github.io>, last access: 17 April 2025) and the Los Alamos Sea Ice Model (**CICE5**) (<https://github.com/COSIMA/cice5/>, last access: 17 April 2025). These components are coupled via the Ocean Atmosphere Sea Ice Soil coupler (**OASIS3-MCT**), developed by CERFACS and CNRS,

France (<https://oasis.cerfacs.fr/en/home/>, last access: 17 April 2025).

The system is driven by the JRA55-do atmospheric reanalysis dataset (Tsuji *et al.*, 2018), through an atmospheric driver called **YATM** (Yet Another Data-Driven Atmosphere Model) (<https://github.com/COSIMA/libaccessom2>, last access: 17 April 2025). The prescribed atmosphere forces the coupled ocean–ice system. A list of exchanged variables between these components enables feedback at every model time step, as shown in Fig. 3.1.

YATM is responsible for tracking model time, reading the appropriate forcing fields from input files, and delivering them to the coupler. It also performs on-the-fly remapping of river runoff data during the simulation.

The model includes an optional biogeochemical module implemented via the World Ocean Model of Biogeochemistry and Trophic-dynamics (**WOMBAT**). This component supports simulations of ocean biogeochemistry and essential for assessment of the ocean’s role in the global carbon cycle.

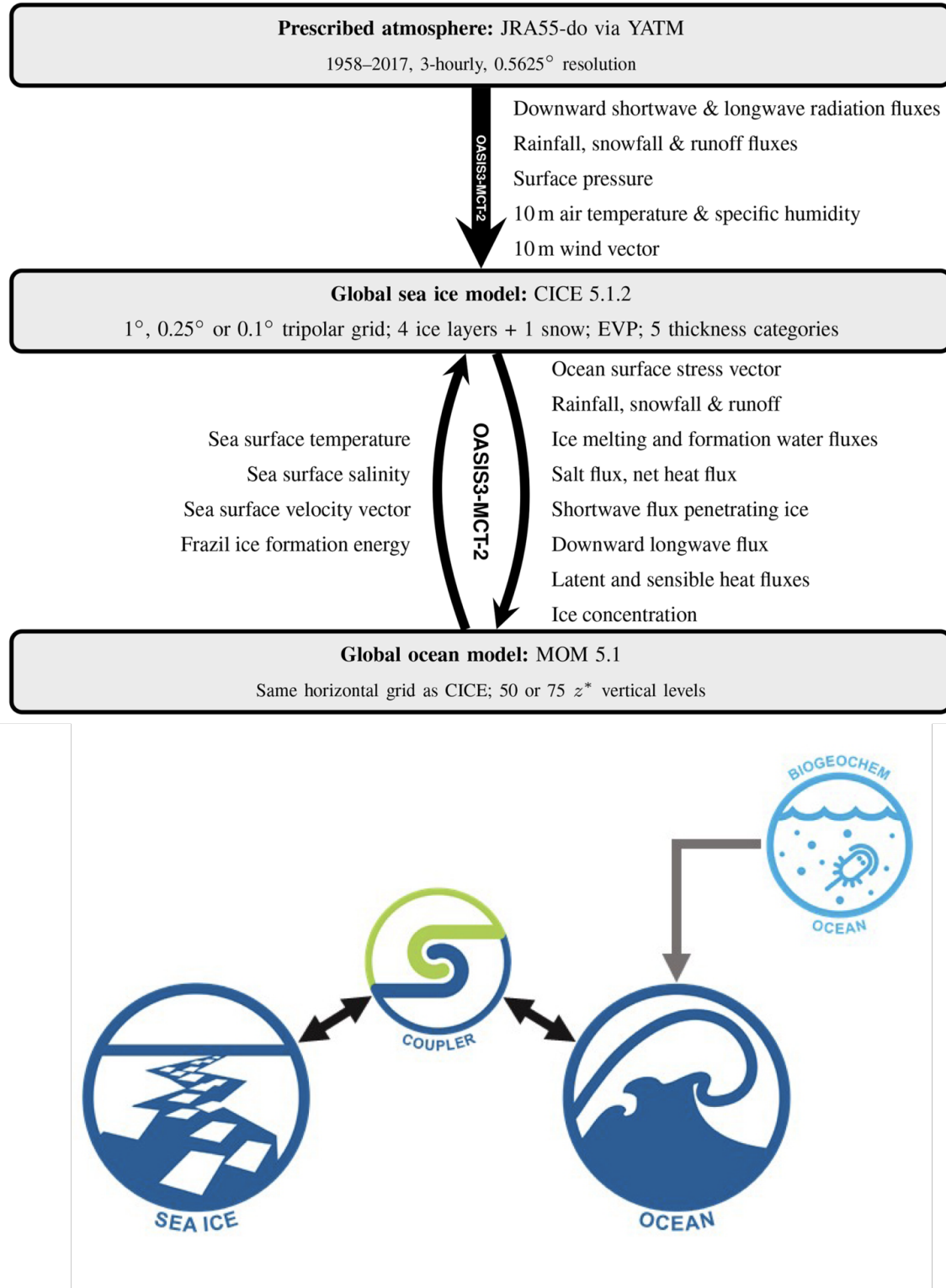


Figure 3.1: ACCESS-OM2 model components and coupling fields. MOM 5.1 receives atmospheric forcing via CICE 5.1.2 rather than directly from YATM (CICE 5.1.2 is configured with the same global domain and horizontal grid as MOM 5.1). Surface pressure is passed from CICE 5.1.2 to MOM 5.1 but is not used by MOM 5.1 in the current configuration so is not shown. Similarly, the sea surface slope vector is passed from MOM 5.1 to CICE 5.1.2 but is unused (the sea surface slope is instead calculated from the sea surface velocity vector, assuming geostrophy). Original Fig. 1 in (Kiss et al., 2020). Schematic representation of ACCESS-OM2 blocks provided by ACCESS-NRI.

While extensive technical aspects and configuration of the model are available in (Kiss et al., 2020) and comprehensive documentation for ACCESS-OM2¹, there are several important features relevant to this study that should be mentioned.

Coupling between atmosphere and sea ice is determined by the temporal resolution of atmospheric forcing (e.g., 3-hourly for JRA55-do). MOM5 and CICE5 exchange fields every time step (5400 seconds in this study). Since both use the same grid, no remapping is needed between them. Conservative interpolation is applied to remap atmospheric fields to the model grid—second-order for 1° and 0.25° , first-order for 0.1° .

This frequent exchange of surface fields (e.g., rainfall, heat flux, surface velocity) presents a performance challenge on HPC systems due to communication overhead and I/O bottlenecks. For example, a double-precision floating point variable of an ocean model over a grid of 0.1° horizontal resolution and 75 vertical levels will typically require over 5 GiB of memory per field. Over 20 such fields are necessary to capture the model state and preserve bitwise reproducibility, and the periodic storage of model output may involve a similar number of variables per diagnostic time step. The model saves NetCDF output files every 24 hours of model time—i.e., with daily temporal resolution. While the computational performance of a coupled model intuitively depends upon the runtime of each of its component models, the I/O process often takes longer than the computation itself. A typical one-year simulation can require reading hundreds of gigabytes of input data and can produce terabytes of model output. For disk speeds of 350 MB/s, a serial transfer of each terabyte would take approximately one hour and severely burden the model runtime.

To improve scalability, recent versions adopt **parallel I/O** (Yang et al., 2020) (Fig. 3.2), which allows multiple processes to read and write NetCDF files simultaneously. Yang et al. (2020) reported up to $60\times$ faster write speeds over serial I/O, a critical advancement for high-resolution simulations

In the 1° setup in this study, the global grid (360×300) is decomposed into smaller 2D blocks ($n_x_block \times n_y_block$) assigned to MPI ranks. Each block contains:

¹<http://cosima.org.au/wp-content/uploads/2020/03/ACCESS-OM2-1-025-010deg-2020-03-12-02bf205.pdf>

- A computational domain (interior grid cells),
- Surrounding halo (ghost) cells for boundary data exchange,

Data distribution is managed via `global_scatter` and `global_gather` routines. Even in single-rank mode, these routines apply consistent indexing to facilitate parallelization.

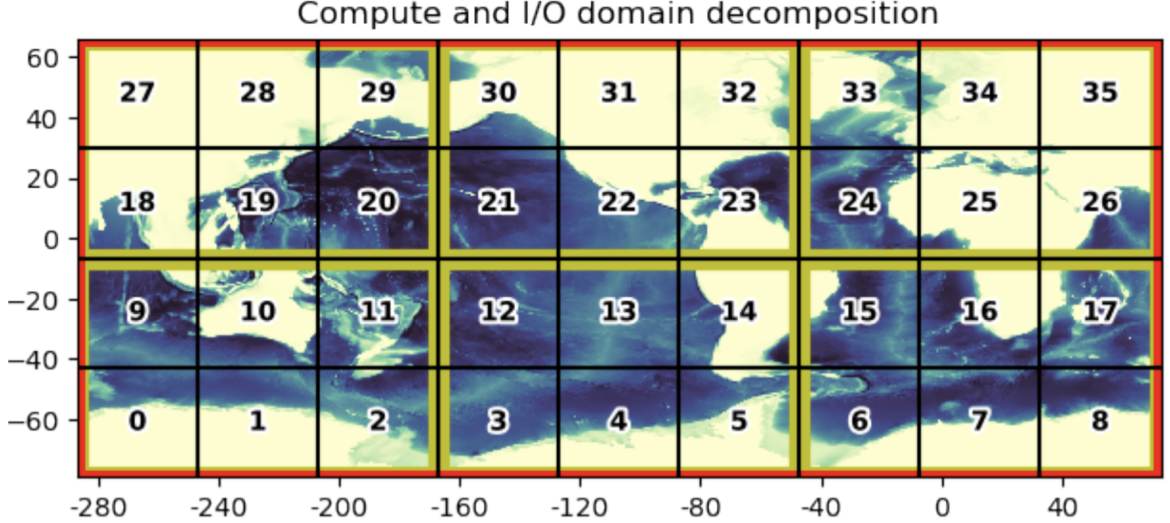


Figure 3.2: Schematic of domain decomposition and parallel I/O. Each black square is a computational tile; yellow regions show I/O aggregators. Adapted from [Yang et al. \(2020\)](#).

3.2 Technical Migration to Leonardo: Compilation and Setup

This section documents the process of porting the ACCESS-OM2 from the Gadi Supercomputer to Leonardo Supercomputer. The migration required adaptations to the model’s codebase, dependencies, and workflow to address architectural disparities, software stack variations, and performance optimizations specific to Leonardo. Below, we detail the challenges, solutions, and validation procedures employed to ensure functional and numerical consistency post-migration. Overall strategy of ACCESS-OM2 migration is summarized in Fig. 3.3.

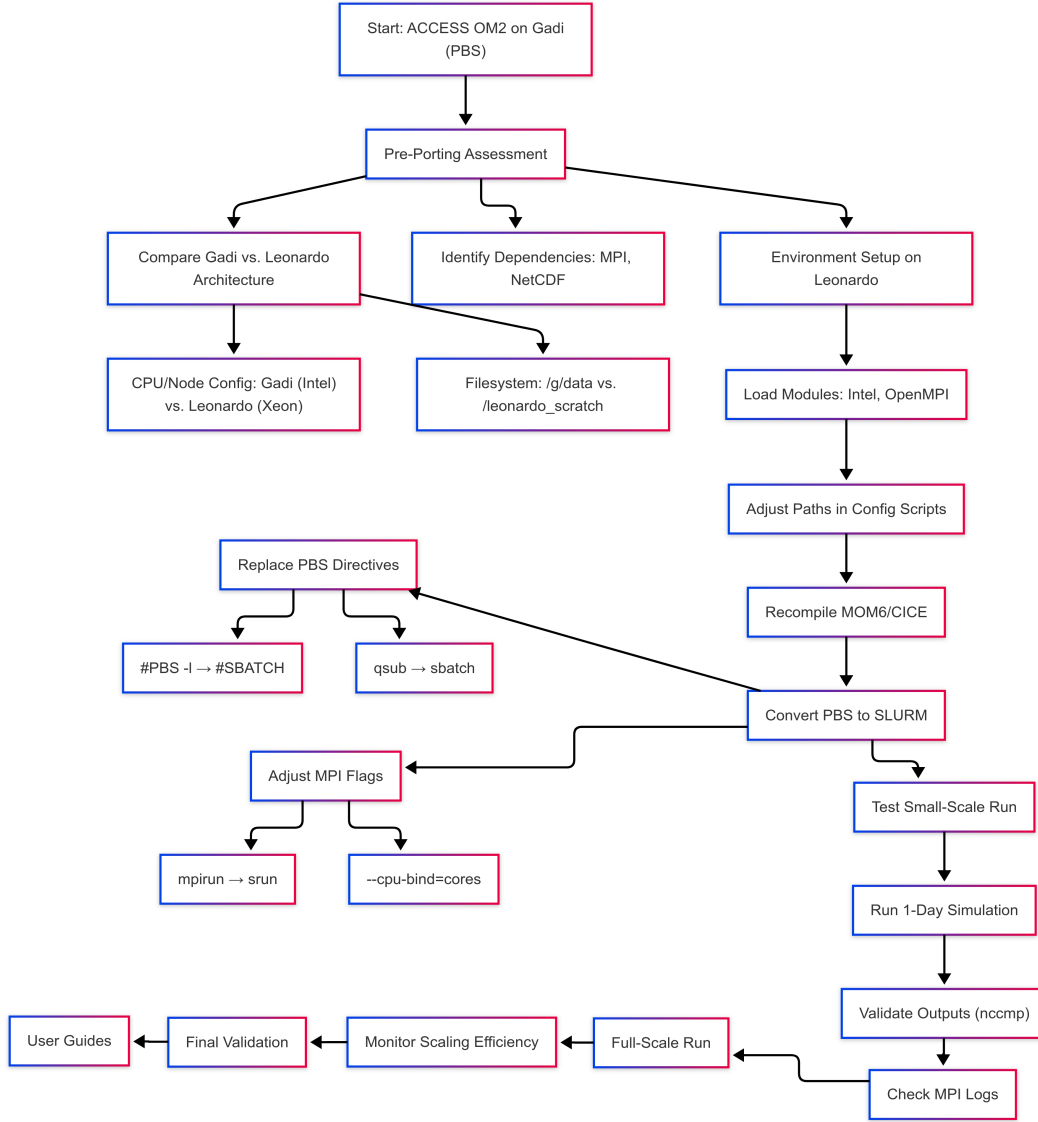


Figure 3.3: Schematic of ACCESS OM2 migration from Gadi to Leonardo.

3.2.1 Initial Conditions and Repeat Year Forcing

To perform any simulation with ACCESS-OM2, a set of initial condition files and prescribed atmospheric forcing data must be provided. This is essentially the physical state of the ocean–sea ice system from which the model begins. The ACCESS-OM2 1° configuration is initialized with the ocean at rest, featuring zero horizontal velocity and zero sea level anomaly. Temperature and salinity fields are taken from the World Ocean Atlas 2013 version 2 (WOA13v2) (Locarnini et al., 2013; Zweng et al., 2013), specifically using the 0.25° “decav” product, which represents the average of six decadal climatologies from 1955 to 2012.

Initial sea ice conditions are defined by full coverage (100% concentration) and a uniform thickness of approximately 2.5 m in regions poleward of 70°N and 60°S, where the sea surface temperature is less than 1°C above freezing. Ice is distributed across the model’s five thickness categories following a parabolic profile. Snow thickness is set to 0.2 m or 20% of the local ice thickness, whichever is smaller. These initial conditions are chosen to closely approximate the model’s equilibrated state, thereby minimizing initial freshwater flux imbalances and avoiding spurious salinity drift during the early years of integration.

In this study, the initial conditions files were not regenerated from scratch. Instead, a consistent and community-validated set of restart files was kindly provided by members of the ACCESS-NRI support team and the COSIMA community. Their assistance significantly reduced the time required for model initialization and ensured compatibility with standard configurations used across prior ACCESS-OM2 simulations.

The simulation conducted in this study employed a Repeat Year Forcing (RYF) dataset based on the JRA55-do atmospheric reanalysis product (Tsuji *et al.*, 2018). The RYF technique involves selecting a single climatological year (typically 1990 or 1991) and repeating it cyclically to generate a temporally homogeneous but dynamically realistic surface forcing dataset. This approach is commonly used in long control simulations, model tuning, and sensitivity experiments, as it allows the ocean–sea ice system to evolve without being influenced by interannual variability in the atmosphere.

The generation of the RYF dataset involved downloading the standard JRA55-do forcing fields from the Japan Meteorological Agency’s official archive (<https://climate.mri-jma.go.jp/pub/ocean/JRA55-do/>). This dataset provides a consistent, high-resolution reanalysis suitable for coupled ocean–sea ice simulations. A custom preprocessing script was then adapted to extract the years 1990–1991 and construct a repeat-year cycle starting in late May 1990 through early June 1991—a period characterized by climatological neutrality in major atmospheric modes such as ENSO and the Southern Annular Mode. Aligning the start of the forcing cycle to this

period helps minimize discontinuities in surface fluxes and reduces the likelihood of “model shock”, a common issue when models are initialized with abrupt transitions in atmospheric conditions.

3.2.2 Model Compilation and Platform Adaptation

The compilation strategy for ACCESS-OM2 on the Gadi supercomputer, at the beginning of this work, was tightly coupled to specific versions of the compiler toolchain and scientific libraries. In particular, the build on Gadi relied on `openmpi-4.1.4`, precompiled with `intel-2021.2.0`, as well as NetCDF libraries, specifically `netcdf-c/4.7.4` and `netcdf-fortran/4.5.2` precompiled with `intel-2021.2.0`. In addition to these, a wide range of system-wide dependencies - such as `automake-1.16.5`, `findutils-4.9.0`, `libevent-2.1.12`, and `yaml-cpp-0.8.0` were a part of the software environment on Gadi. These utilities, typically standard across Linux-based HPC systems, numbered close to 100 and included a mix of development tools, system libraries, and auxiliary packages.

While most of these tools are intended to be interoperable across HPC platforms, the specific versions and combinations available on different systems can vary substantially. Compilation repeatedly failed due to differences in compiler toolchains and library versions. In particular, Gadi provided OpenMPI built with Intel compilers (e.g., 2021.2.0), while Leonardo’s available modules were compiled with GCC or NVHPC, such as `openmpi/4.1.6-gcc-12.2.0` and `openmpi/4.1.6-nvhpc-24.3`. This mismatch rendered many Intel-specific compiler and linker flags embedded in the build system non-functional.

ACCESS-OM2’s build scripts include numerous hardcoded flags tailored to Intel compilers—covering optimization, debugging, and floating-point behavior—that were incompatible with Leonardo’s environment. The following snippet illustrates some of these flags:

Listing 3.1: Example of compiler-dependent flags in ACCESS-OM2 build system

```
FPPFLAGS      := -fpp -Wp,-w $(INCLUDE)
```

```

FFLAGS      := -fno-alias -safe-cray-ptr -fpe0 -ftz -assume byterecl -i4 -r8
              -traceback -nowarn -check noarg_temp_created -assume nobuffered_io
              -convert big_endian -grecord-gcc-switches -align all

FFLAGS_OPT   := -g3 -O2 -xCORE-AVX2 -debug all -check none
FFLAGS_DEBUG := -g3 -O0 -debug all -check -warn -ftrapuv
FFLAGS_REPRO := -fp-model precise -fp-model source -align all

CFLAGS      := -D__IFC $(INCLUDE)
CFLAGS_OPT   := -O2 -debug minimal -xCORE-AVX2
CFLAGS_DEBUG := -O0 -g -ftrapuv -traceback

LDFFLAGS     :=
LDFFLAGS_VERBOSE := -Wl,-V,--verbose,-cref,-M

```

Resolving these issues required extensive manual refactoring of the build process. This included iterative compilation of individual components, detailed inspection of error logs, and systematic replacement or removal of incompatible flags. Though time-consuming, completing this task enabled successful model compilation on Leonardo and yielded valuable insights into the fragility of tightly coupled build systems when transitioning across heterogeneous HPC architectures. The listing presents the compiled executables corresponding to each model component, including multiple resolution-specific versions of the CICE5 sea ice model, the YATM atmospheric driver, the MOM5 ocean model coupled via OASIS3-MCT, and the postprocessing utility mppnccombine.

Listing 3.2: Compiled binaries in the COSIMA-based ACCESS-OM2 repository. Includes multiple resolution-specific CICE executables, YATM, MOM+OASIS, configuration for different resolutions and forcings, and other tools.

```

ACCESS-OM2-COSIMA
|-- .github/workflows
|-- bin
|   |-- cice_auscom_18x15.3600x2700_1682p_f39c802_libaccessom2_7ce7920.exe
|   |-- cice_auscom_360x300_24p_f39c802_libaccessom2_7ce7920.exe
|   |-- cice_auscom_1440x1080_480p_f39c802_libaccessom2_7ce7920.exe
|   |-- cice_auscom_3600x2700_722p_f39c802_libaccessom2_7ce7920.exe
|   |-- fms_ACCESS-OM_1883a84_libaccessom2_7ce7920.x
|   |-- mppnccombine
|   |-- yatm_7ce7920.exe
|-- control
|   |-- 1deg_jra55_iaf
|   |-- 01deg_jra55_iaf
|   |-- 1deg_jra55_ryf
|   |-- 01deg_jra55_ryf
|   |-- 025deg_jra55_iaf
|   |-- 025deg_jra55_ryf
|-- doc
|-- src
|   |-- cice5
|   |-- libaccessom2
|   |-- mom

```



```

|-- test
|-- tools
|-- .gitignore
|-- .gitmodules
|-- accessom.yml
|-- environment.sh
|-- hashexe-public.sh
|-- hashexe.sh
|-- install.sh
|-- libcheck.sh
|-- MOM_diags.txt
|-- output.txt
|-- README.md

```

Table 3.1: File count per top-level directory in the `access-om2` repository

Location	Files	Notes / File Types
src/	7,697	Large codebase: Fortran/C source files, headers, build configs
.git/	1,070	Internal Git metadata (not relevant for compilation)
control/	335	Configuration sets for experiments, YAML and namelist files
tools/	92	Utility scripts for diagnostics and setup (mostly shell)
test/	18	Basic testing or validation configurations
bin/	7	Compiled binaries
README.md	1	Project overview in Markdown format
output.txt	1	Log file
MOM_diags.txt	1	Diagnostics from the MOM component
libcheck.sh	1	Shell script for library checking
install.sh	1	Model installation shell script
hashexe.sh	1	Utility for hashing executables
hashexe-public.sh	1	Public-facing version of hashing tool
.gitmodules	1	Git submodules configuration
.gitignore	1	Git ignore file
.github/	1	GitHub Actions/workflows directory
environment.sh	1	Environment variable setup script
doc/	1	Documentation placeholder
accessom.yml	1	Main YAML configuration for the model

The resulting model executables (e.g., CICE, fms_ACCESS-OM, YATM) are dynamically linked to hundreds of intermediate files generated during the build process. For each component, compilation produces module files (*.mod) and object files (*.o), that reflect the structure and subroutine-level modularization of the codebase.

Nearly every physical process or computational task encapsulated in a dedicated Fortran module. These components are modules for e.g. ice thermodynamics (ice_therm_bl99.f90, ice_therm_0layer.f90), radiative forcing (ice_shortwave.f90), and state handling (ice_state.f90), among others.

```
cice/build/
|-- ice_restart.mod
|-- ice_restoring.mod
|-- ice_shortwave.mod
|-- ice_spacecurve.mod
|-- ice_state.mod
|-- ice_therm_bl99.mod
...
```

This modular structure, while enabling maintainability and code reuse, significantly increases the number of compilation outputs.

Toward the end of this compilation work, the COSIMA repository - previously the main development and distribution platform for ACCESS-OM2 was **officially deprecated**. A deprecation notice was issued, stating that the ACCESS-National Research Infrastructure (ACCESS-NRI) had assumed responsibility for the continued development, maintenance, and support of ACCESS-OM2. As a result, the COSIMA-based codebase would no longer be updated and was slated for archival.

The updated development strategy shifted toward a centralized and more formalized workflow under ACCESS-NRI, utilizing new repositories such as <https://github.com/ACCESS-NRI/ACCESS-OM2> and <https://github.com/ACCESS-NRI/access-om2-configs>. This transition marked a move from community-driven maintenance to coordinated support by a dedicated team of developers, including contributors with backgrounds in core Linux kernel development. In light of these changes, users were strongly encouraged to migrate to the new ACCESS-NRI-supported workflow to ensure compatibility with future developments, reproducibility standards, and long-term maintainability. Accordingly, we proceeded to transition the model setup to the new ACCESS-NRI configuration and installation framework.

3.2.3 Compilation Using the ACCESS-NRI Spack-Based Strategy

Following the deprecation of the COSIMA-maintained ACCESS-OM2 repository, responsibility for model support transitioned to ACCESS-NRI. Alongside this shift, a new installation and compilation workflow was introduced, emphasizing modularity, reproducibility, and portability through the use of the Spack package manager. The ACCESS-NRI team recommends an “all-packed” approach, in which the entire compilation toolchain — including compilers, MPI libraries, and dependencies — is built and managed locally within the user space using Spack.

This workflow begins with the local installation of Spack, followed by the configuration of additional source-code repositories for Spack packages maintained by the ACCESS-NRI team. These packages are hosted in a dedicated repository available at: github.com/ACCESS-NRI/spack-packages. The recipes are version-controlled, actively maintained, and integrated with continuous integration workflows provided by the ACCESS-NRI `build-ci` repository. This CI system supports automated JSON validation, package building, and dependency image generation triggered by version-tagged commits.

Listing 3.3: Full ACCESS-NRI Spack installation and environment setup

```
# Clone Spack and configuration repositories
git clone -c feature.manyFiles=true https://github.com/spack/spack.git
--branch releases/v0.20 --single-branch --depth=1
git clone https://github.com/ACCESS-NRI/spack-packages.git --branch main
git clone https://github.com/ACCESS-NRI/spack-config.git --branch main

# Link configuration files
ln -s -r -v spack-config/v0.20/ci/* spack/etc/spack/
. spack-config/spack-enable.bash
```

Alongside the default open-access Spack source repository, an additional custom repository maintained by ACCESS-NRI (`access.nri`) is also added to the configuration:

```
[ntilinin at login01 ACCESS-NRI] spack repo list
==> 2 package repositories.
access.nri    /leonardo/home/userexternal/ntilinin/ACCESS-NRI/spack-packages
builtin      /leonardo/home/userexternal/ntilinin/ACCESS-NRI/spack/var/spack/
repos/builtin
```

Once Spack and the necessary repositories were configured, the installation proceeded with the Intel compiler suite using Spack, initially bootstrapped via the system-wide GCC compiler. Subsequently, OpenMPI was compiled using the Intel toolchain, and the NetCDF libraries were built on top of the resulting MPI stack. All components were installed within a Spack-managed environment, ensuring strict version consistency across the entire toolchain.

It is important to note that the version of OpenMPI used in this installation was built locally with Intel compilers. This contrasts with the default system-wide installation of OpenMPI on Leonardo, which is compiled with GCC and therefore incompatible with the Intel-based toolchain required for ACCESS-OM2. Specifically, the ACCESS-OM2 installation continued as:

```
spack install intel-oneapi-compilers@2021.2.0 target=x86_64

spack load intel-oneapi-compilers@2021.2.0
spack compiler find

spack install access-om2 ^netcdf-c@4.7.4 ^netcdf-fortran@4.5.2
^parallel-io@2.5.2 ^openmpi@4.0.2 %intel@2021.2.0 target=x86_64

spack find
```

These versions match the current ACCESS-OM2 configuration and ensure compatibility across all model components including MOM5, CICE5, YATM, OASIS coupler and other components.

The Spack environment also generates separate Tcl module files for each installed component, including individual module called `access-om2` with linked modules for MOM5, CICE5, YATM and all other dependent modules. This modularization enables users to selectively load or test components within controlled environments, but it also reflects the granularity of Spack's packaging approach:

```
[ntilinin@login01 ACCESS-NRI]$ module avail
access-om2/access-om2-o33fig
intel-oneapi-compilers/2021.2.0-p7vtyvv
mom5/access-om2-ogthsyx
netcdf-fortran/4.5.2-5ln2rw2
openmpi/4.1.4-ga6avsd
pmix/4.2.2-ynryh6y
cice5/access-om2-woizzl5
libaccessom2/access-om2-lnlbi4d
netcdf-c/4.7.4-d4klyvu
oasis3-mct/access-om2-ltccmcj
parallel-io/2.5.2-fmgl254
totalview/2024.3-x86-64-gjk4f4g
...
```

```
etc.
```

With components of ACCESS-OM2 available system wide as modules:

```
[ntilinin@login01 ACCESS-NRI]$ which cice_aucom_360x300_24x1_24p.exe
~/ACCESS-NRI/release/linux-rhel8-x86_64/intel-2021.2.0/
cice5-access-om2-woizzl5sdrgejgmrvt7gax5gmrxxgj/bin/
cice_aucom_360x300_24x1_24p.exe
```

Although this approach ensures compatibility and isolation, it may introduce performance trade-offs. In particular, the model executables built using Spack-installed compilers and libraries (e.g., Intel 2021.2.0, OpenMPI 4.0.2, NetCDF 4.7.4) are not specifically tuned for the Leonardo HPC system’s architecture. In contrast, executables compiled directly against the system-wide software stack - typically optimized by the site administrators for the underlying hardware - are likely to exhibit superior runtime performance. While this hypothesis was not explicitly benchmarked in this study, it is a relevant consideration for future production deployments where compute efficiency is critical.

This strategy also introduces limitations. Since the compilation is performed through Spack recipes maintained by ACCESS-NRI, the build process abstracts away most low-level control, including the ability to directly set compiler optimization flags (e.g., `-O3`, `-xHost`, `-ipo`) or to modify Makefiles and build scripts used by individual model components. These flags are often critical for tailoring performance to the specific architecture of a given HPC system. While this approach promotes reproducibility and simplifies installation, it reduces flexibility for system-specific tuning or advanced performance optimization during model porting. Local modification of Spack build recipes is possible but requires additional maintenance effort and synchronization with general model development as well as careful version management to avoid breaking compatibility.

Troubleshooting OpenMPI and Infiniband Configuration

Installation of ACCESS-OM2, along with required dependencies such as `intel-oneapi-compilers`, `OpenMPI`, `NetCDF`, and `ParallelIO`, while conceptually straightforward, revealed several challenges during execution. While many minor

issues were easily resolved along the way, two major complications required detailed investigation and workarounds.

One key architectural concern was whether a user-space installation of OpenMPI would function properly on a large HPC system like Leonardo, where high-performance networking and message-passing behavior typically rely on site-managed libraries and hardware drivers (e.g., MVAPICH). Nevertheless, all ACCESS-OM2 components — including MOM5, CICE5, YATM, and the OASIS coupler — were successfully compiled using a fully local Spack-based environment.

However, initial runtime tests with even the simplest MPI programs (`mpi_hello_world.x`) failed, as did test launches of the full model. The failures included communication deadlocks and task launching errors:

Listing 3.4: SLURM task launch error

```
srun: error: Task launch for StepId=12863828.0 failed on node lrdn4190:
Communication connection failure
srun: error: Task launch for StepId=12863827.0 failed on node lrdn4735:
Communication connection failure
srun: error: Task launch for StepId=12863828.0 failed on node lrdn4184:
Communication connection failure
...
```

Together with:

Listing 3.5: OpenMPI endpoint error

```
[lrdn4329] [[60295,30638],225] [btl_tcp_endpoint.c:626:
mca_btl_tcp_endpoint_recv_connect_ack]
received unexpected process identifier [[60295,30638],273]

[lrdn4315] [[60295,30638],1] [btl_tcp_endpoint.c:626:
mca_btl_tcp_endpoint_recv_connect_ack]
received unexpected process identifier [[60295,30638],4]

[lrdn4334] [[60295,30638],337] [btl_tcp_endpoint.c:626:
mca_btl_tcp_endpoint_recv_connect_ack]
received unexpected process identifier [[60295,30638],352]
...
```

Analysis of the error logs and extensive OpenMPI documentation revealed two primary problems:

1. MPI programs were running exclusively on a single node, even when multiple nodes were requested — indicating OpenMPI was not properly integrated with SLURM.

2. Repeated `unexpected process identifier` errors from `btl_tcp_endpoint.c` suggested that inter-process communication via TCP transport layer (`btl_tcp` stands for Byte Transfer Layer using TCP) was being attempted across nodes but failing due to packet loss or incorrect routing.

Issue 1: OpenMPI Not Compiled with SLURM Support

Upon examining the default OpenMPI Spack configuration provided by ACCESS-NRI, it was discovered that OpenMPI had been built without SLURM support (`-without-slurm`), as Gadi internal systems are PBS-based. This led to job launches silently ignoring node distributions and defaulting to local-only execution.

To resolve this, OpenMPI was recompiled via Spack using the proper SLURM integration flags, matching the SLURM-enabled configuration used by Leonardo's system-wide :

```
spack install access-om2 ^netcdf-c@4.7.4 ^netcdf-fortran@4.5.2
^parallel-io@2.5.2 ^openmpi@4.1.4 +lustre ~memchecker +pmi ~static
~wrapper-rpath fabrics=ucx schedulers=slurm %intel@2021.2.0 target=x86_64
```

Two critical options:

- `fabrics=ucx` enables support for the Unified Communication X (UCX) library, which provides low-latency, high-bandwidth communication over Infiniband, RoCE, or shared memory. This is essential for maximizing message-passing performance on HPC systems such as Leonardo.
- `schedulers=slurm` ensures integration with the SLURM workload manager, allowing OpenMPI to correctly interface with `srun` and `mpirun` for launching distributed jobs across multiple compute nodes.

Issue 2: InfiniBand Not Used by Default

After resolving the SLURM issue, another problem persisted:

```
[1r4n4334] [[60295,30638],337] btl_tcp_endpoint.c:626:
mca_btl_tcp_endpoint_recv_connect_ack]
received unexpected process identifier [[60295,30638],352]
```

Investigation revealed that likely OpenMPI defaulted to **Ethernet** rather than using Leonardo’s **InfiniBand** high-speed interconnect. At the same time the `ibstat` tool confirmed available high-performance interface:

```
CA 'mlx5_4'
...
Port 1:
    State: Active
    Physical state: LinkUp
    ...
    Port GUID: 0x043f720300d30db6
    Link layer: InfiniBand
```

Out of several interfaces (`mlx5_1`, `mlx5_bond_0`), only `mlx5_4` had an active InfiniBand link. This confirmed that `mlx5_4` was the correct interface to be used for low-latency communication. Subsequent configuration of OpenMPI to utilize this port via environment variables (e.g., `OMPI_MCA_btl_tcp_if_include=ib0`) ensured that model execution utilized the appropriate high-speed interconnect:

Listing 3.6: MPI environment configuration for I/O tuning and interface selection

```
export OMPI_MCA_btl_tcp_if_include=ib0
export MCA_IO=ompio
export MCA_IO_OMPIO_NUM_AGGREGATORS=1
```

With the appropriate runtime MCA parameters set inter-node communication was successfully routed over the InfiniBand fabric, and model execution scaled efficiently across multiple compute nodes.

Although this debugging process extended over several weeks, the final outcome was a fully functional and portable software stack built entirely within user space — sufficient to compile all ACCESS-OM2 components (YATM, CICE5, and MOM5) using consistent, compatible compiler and library versions. This effort demonstrates both the flexibility and complexity of modern reproducible HPC workflows, and highlights the importance of environment-level compatibility when porting large coupled models across supercomputing architectures.

3.3 Adjustment of the Payu Workflow Manager for SLURM Job Submission

The `payu` workflow manager is widely used within the ACCESS-OM2 ecosystem to orchestrate model runs, manage input and output files, and handle job submissions across high-performance computing (HPC) environments. Originally developed for systems like Gadi, which use the PBS Pro scheduler, `payu` required targeted modifications to operate correctly with SLURM — the scheduler employed on Leonardo and many other modern HPC platforms. The `payu` repository includes scripts for environmental setup and core components responsible for platform module management, experiment configuration, laboratory creation for individual experiments, metadata handling, GitHub hashing and more:

```
....
|-- conda
|   |-- environment.yml
|   |-- meta.yaml
...
|-- payu
|   |-- backports.py
|   |-- branch.py
|   |-- calendar.py
|   |-- cli.py
|   |-- envmod.py
|   |-- experiment.py
|   |-- fsops.py
|   |-- git_utils.py
|   |-- __init__.py
|   |-- laboratory.py
|   |-- manifest.py
|   |-- metadata.py
|   |-- models
|       |-- accessom2.py
|       |-- cable.py
|       |-- cice.py
|       |-- fms.py
|       |-- mom.py
|       |-- yatm.py
|   |-- namcouple.py
|   |-- profilers
|       |-- darshan.py
|       |-- profiler.py
...
|-- setup.py
|-- test
|   |-- common.py
|   |-- models
|       |-- test_access.py
|   |-- resources
|       |-- config_modules.yaml
|       |-- sample_ldd_output.txt
```

The core modifications centered around the management of modules environment

and job submission logic, which is defined in Payu’s configuration and run files (e.g., `config.yaml`, `envmod.py`, `experiment.py`, `laboratory.py`) and in its Python-based backend. Key changes included updating the job submission templates, adjusting resource request parameters (e.g., `-ntasks-per-node`, `-cpus-per-task`), and ensuring compatibility with SLURM’s environment variable handling. The PBS-specific directives were systematically replaced with SLURM equivalents:

```
#PBS -N → #SBATCH -job-name
#PBS -l walltime → #SBATCH -time
#PBS -l mem → #SBATCH -mem
#PBS -q → #SBATCH -partition
```

The original PBS task distribution mechanism was replaced with SLURM’s native MPI support:

```
#SBATCH -nodes=4
#SBATCH -ntasks-per-node=32
#SBATCH -cpus-per-task=2
```

Modifications also included environment module handling and customizing the runtime execution logic to generate SLURM-compliant job commands using ‘`srun`’ instead of ‘`mpirun`’ (used with PBS on systems like Gadi). SLURM-specific arguments (e.g., `--chdir`, `--exact`, etc.) were programmatically injected based on each model component’s configuration.

These targeted adjustments enabled reproducible model runs on SLURM-based clusters without altering the core `payu` logic designed for PBS compatibility. Optional runtime profiling was also explored through the integration of `perf` and `mpiP` instrumentation wrappers. These changes were implemented within the SLURM scheduler class and related interfaces across the `payu` codebase:

```
1 if self.config.get('scheduler') == 'slurm':
2     # SLURM-specific runtime flags
3     model_prog.append('-n {0}'.format(model_ncpus))
4     add_arg = '--exact'
5     model_prog.append(f'{add_arg}')
6     model_prog.append(f'--partition={0}'.format(model_partition))
7     wdir_arg = '--chdir'
8     model_prog.append(f'{wdir_arg} {model.work_path}')
```

```

9     model_prog.append('--nodes={0}'.format(model_nodes))
10    model_prog.append('--ntasks-per-node={0}'.format(model_tasks_per_node))
11
12    # Example for optional profiling hooks
13    mpiP_dir = os.path.join(self.work_path, 'mpiP_perf_profiles')
14    os.makedirs(mpiP_dir, exist_ok=True)
15    f_out_mpiP = open(os.path.join(mpiP_dir, f'{model.model_type}.mpiP.prof'), 'w')
16    f_out_perf = open(os.path.join(mpiP_dir, f'{model.model_type}.perf.prof'), 'w')
17    f_out_mpiP.write("Some data related to mpiP profile")
18    f_out_perf.write("Some performance data")
19
20    # Example environment and perf wrapping (currently commented out)
21    mpi_env_vars = (
22        f'env MPIP_ARG_LOGFILE={f_out_mpiP.name} '
23        'MPIP_ARG_LOGFILE_TYPE=merged '
24        'MPIP_ARG_TIMING_THRESH=0.1 '
25        'MPIP_ARG_MSG_SIZE=1 '
26        'MPIP_ARG_REPORT=full '
27        'MPIP_ARG_TRACELEVEL=1 '
28        'MPIP_TIME=clock_gettime '
29        'LD_PRELOAD=/path/to/libmpiP.so'
30    )
31    perf_cmd = (
32        f'perf stat -e cycles,instructions,cache-misses,branch-misses,'
33        'context-switches,cpu-migrations,page-faults -- {mpi_env_vars}'
34    )
35    model_prog.append(perf_cmd)

```

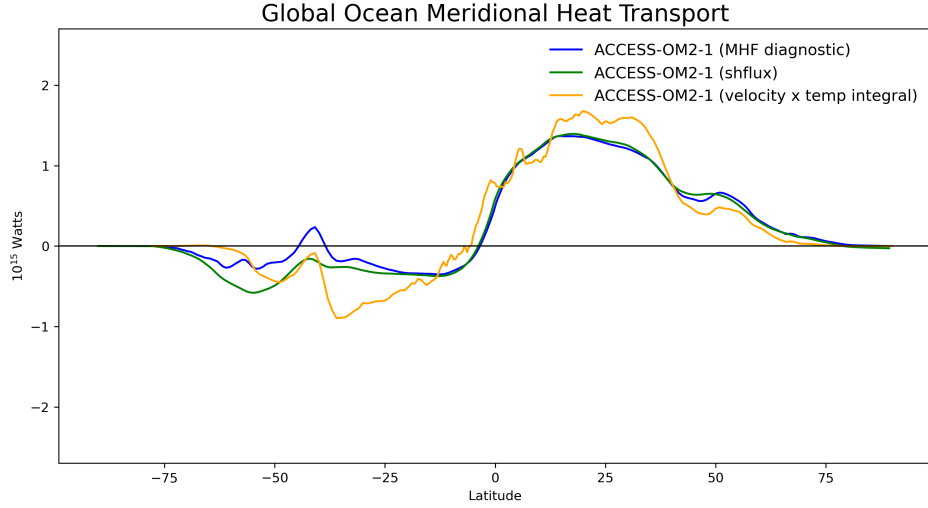
Listing 3.7: SLURM-specific Class configuration block within the payu workflow

3.4 Running Model Experiments and Postprocessing Simulation Output

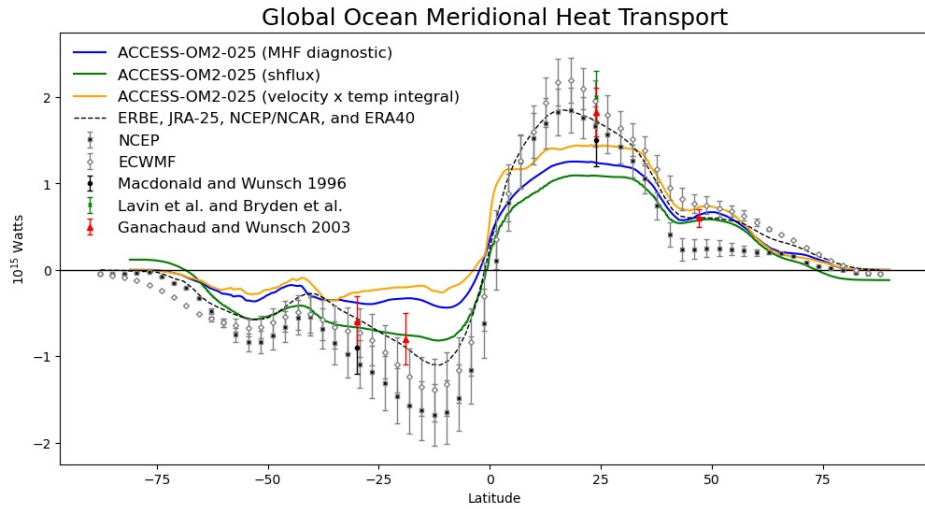
Model output diagnostics were produced from the ACCESS-OM2 simulations in NetCDF format using post-processing tools based on the [COSIMA Recipes](#) analysis framework. One of the key diagnostics evaluated is the **Meridional Heat Transport (MHT)**, which serves as a crucial metric for assessing large-scale ocean circulation and energy redistribution.

The plots presented in Fig. 3.4 below were generated from simulation outputs performed in this study and reproduce the MHT diagnostic using three distinct methods: the native MHF diagnostic, surface heat flux integration (**shflux**), and the velocity–temperature integral method. The resulting figures closely resemble those originally produced by the ACCESS-OM2 development team, validating the consistency of the model run, postprocessing pipeline, and configuration.

The sea ice maps for the Arctic (December 1929) and Antarctic (November 1931), as produced by the model (Figures 3.5 and 3.6), display realistic and physically consistent distributions of sea ice for their respective hemispheres and seasons. In the Arctic, high sea ice concentration is centered over the central Arctic Ocean, with



(a) Meridional Heat Transport from ACCESS-OM2-1 simulation (this study)



(b) Reference plot from ACCESS-OM2-025 simulation (ACCESS-NRI)

Figure 3.4: Comparison of global ocean Meridional Heat Transport (MHT) diagnostics produced in this study (a) and the reference plot from ACCESS-OM2-025 by ACCESS-NRI (b). The three lines in each represent calculations from MHF diagnostic, surface heat fluxes, and velocity-temperature integral.

gradual thinning toward the marginal seas such as the Barents, Kara, Chukchi, and Beaufort. This spatial pattern is in line with modern observational climatology, where multiyear ice dominates the central basin and seasonal ice forms at the periphery.

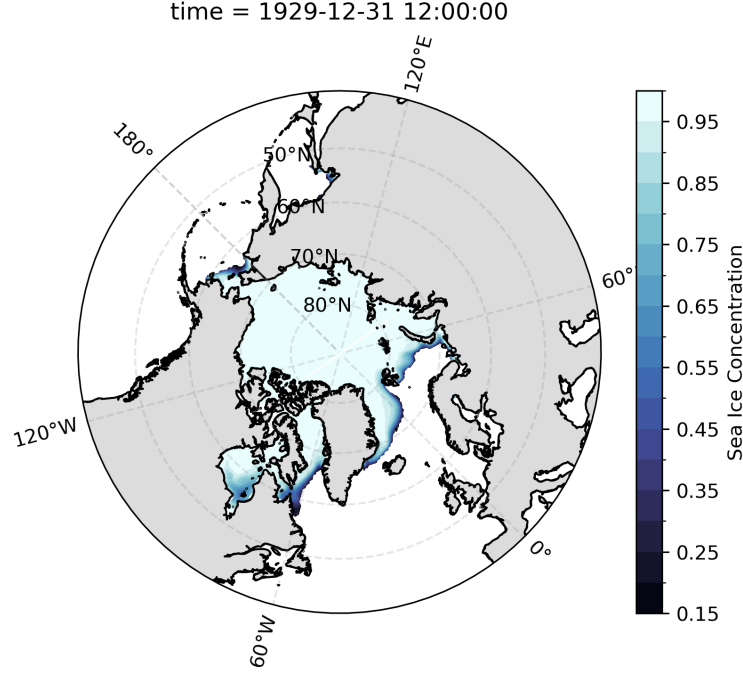


Figure 3.5: Sea Ice Concentration in the Arctic, December 1929.

In the Antarctic, the November 1931 snapshot represents a late austral spring state. The ice edge is seen to hug the coast, consistent with a seasonal retreat after the winter maximum. The asymmetry in sea ice extent and concentration around the continent reflects the influence of bathymetric constraints and ocean circulation, including the Antarctic Circumpolar Current and the formation of coastal polynyas.

The time series of total sea ice area in both hemispheres (Figure 3.7) spans model years 1926 to 1932. These dates do not correspond to actual historical years but rather represent a virtual period within the numerical simulation. Nevertheless, the model reproduces a realistic seasonal cycle: Arctic sea ice area peaks in late winter (around March) and reaches its minimum in September, while Antarctic sea ice follows the opposite phasing, peaking in September and declining during the summer months. The magnitude and phase of these oscillations are consistent with satellite-era observations, indicating that the model is successfully capturing the dominant thermodynamic and dynamic processes governing sea ice variability.

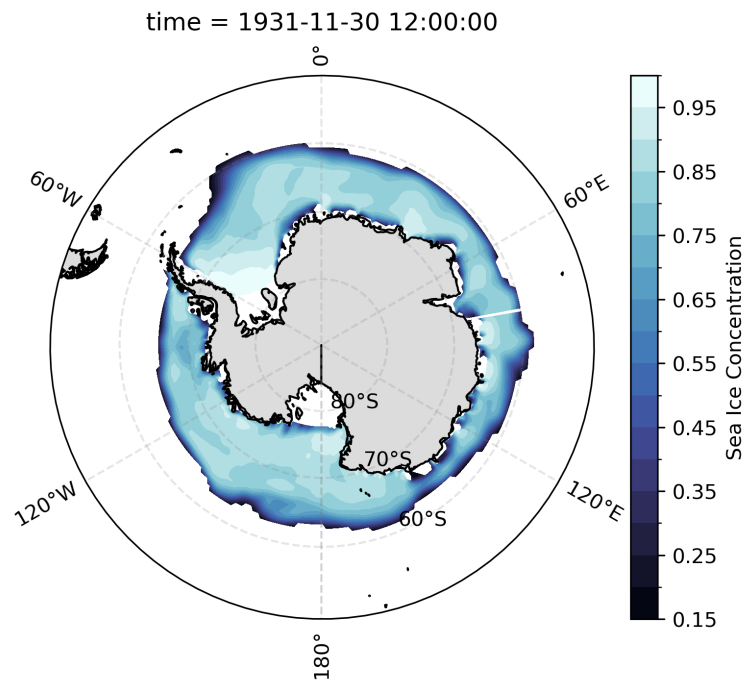


Figure 3.6: Sea Ice Concentration in the Antarctic, November 1931.

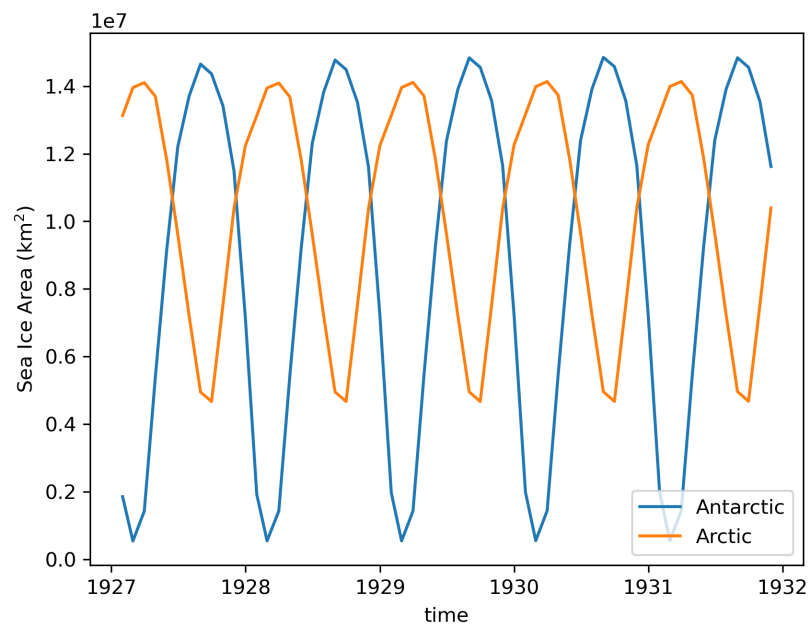


Figure 3.7: Seasonal cycle of total sea ice area in both hemispheres from 1926–1932. Antarctic sea ice area (blue) peaks around September, while Arctic (orange) peaks in March.

Figure 3.8 illustrates a meridional cross-slope section in the South Atlantic. The transect spans from approximately 62°W to 56.5°W, the red line shows the sampling path, aligned with key topographic gradients. Isopycnals slope upward towards the continental shelf, consistent with baroclinic structure and the influence of shelf/slope processes. A sharp density gradient is visible in the upper 100 m, suggesting active surface mixing and potential for water mass transformation. The deepest isopycnals (1037–1038 kg/m³) exhibit minor vertical tilting, hinting at a stable deep water mass configuration.

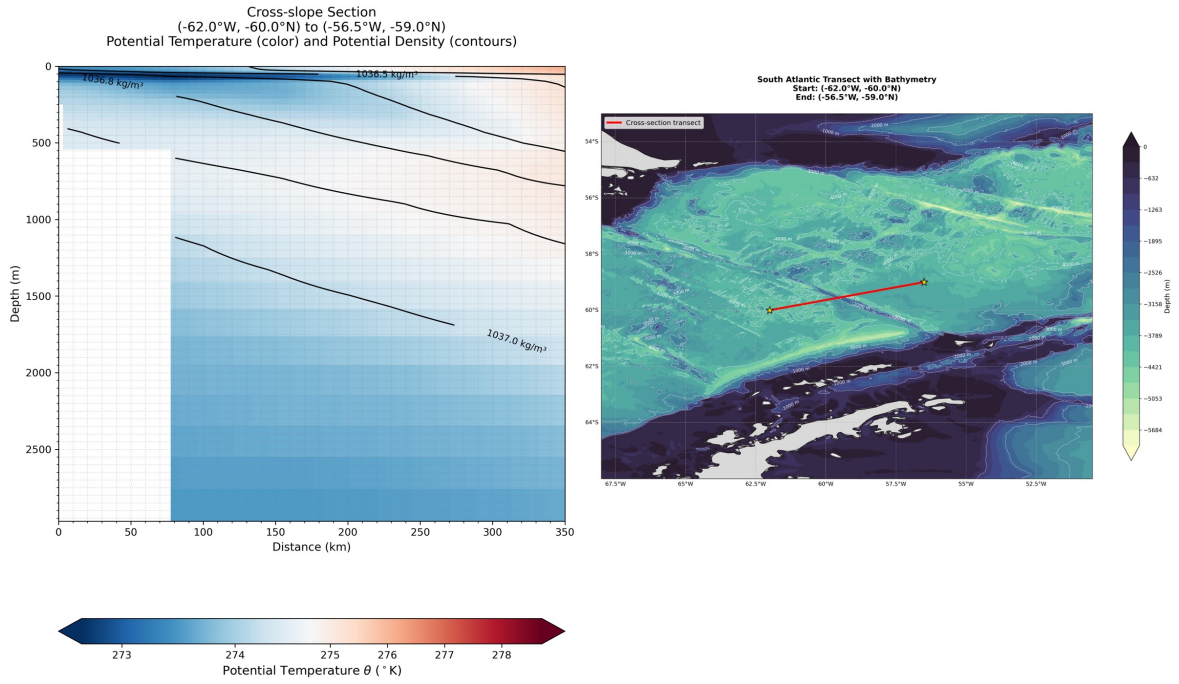


Figure 3.8: Cross-slope stratification and bathymetry in the South Atlantic region.

Overall, figures presented in this section collectively confirm the realism and correctness of the model run.

4 Performance Profiling of ACCESS-OM2 on the Leonardo Supercomputer with perf and mpiP tools

This chapter presents a detailed performance profiling of the ACCESS-OM2 coupled model, ported and deployed on the Leonardo supercomputer. The primary objective is to evaluate computational efficiency, detect component-level bottlenecks, and validate the integration of profiling tools on this HPC platform.

All experiments were conducted using the only currently available fixed configuration of ACCESS-OM2, which is constrained by the availability of an initial conditions dataset provided by the ACCESS-NRI team. This dataset corresponds to the 1° resolution configuration. Running the model at other resolutions or configurations would require generating new initial condition datasets, which involves nontrivial preprocessing pipelines and was considered beyond the scope of this thesis.

The profiling setup employed the standard 1° ACCESS-OM2 configuration, which utilizes a regular global grid of 360×300 cells in longitude and latitude, respectively.

At compilation time, both MOM5 and CICE5 are built with fixed domain decompositions. These decompositions define static block sizes and halo exchange regions, which cannot be adjusted at runtime:

- MOM5 is compiled with a layout of $16 \times 15 = 240$ MPI tiles, of which 216 are actively used due to land masking; the remaining tiles are allocated to land-only subdomains and remain idle.
- CICE5 is configured to run with 24 MPI ranks, using a slender longitudinal decomposition strategy suitable for tripolar grids.

The following code block was integrated into the payu workflow manager to enable lightweight runtime profiling of coupled model runs. It introduces support for perf and mpiP tools, allowing for collection of hardware-level and MPI communication metrics with minimal overhead:

```

# Wrap perf + mpiP3.5 profiling of the coupled model execution
mpi_env_vars = (
f'env MPIP_ARG_LOGFILE={f_out_mpiP.name} '
'MPIP_ARG_LOGFILE_TYPE=merged '
'MPIP_ARG_TIMING_THRESH=0.1 '
'MPIP_ARG_MSG_SIZE=1 '
'MPIP_ARG_REPORT=full '
'MPIP_ARG_TRACELEVEL=1 '
'MPIP_TIME=clock_gettime '
'LD_PRELOAD=/leonardo/home/userexternal/ntilinin/ACCESS-NRI/release/
linux-rhel8-x86_64_v4/gcc-8.5.0/mpip-3.5-wz3kzhgv3lbgdbgw2i45opob4ck2n4v/lib/
libmpiP.so'
)

perf_cmd = (
f'perf stat -e cycles,instructions,cache-misses,branch-misses,
context-switches,cpu-migrations,page-faults '
f'-- {mpi_env_vars}'
)

# Wrap entire command in shell execution string
model_prog.append(perf_cmd)

```

Listing 1: mpiP and perf profiling

4.1 Runtime analysis of MOM5 and CICE5

Figure 4.1 compares the runtime performance of the ACCESS-OM2 1° configuration on Leonardo (highlighted in cyan) against benchmark results from [Kiss et al. \(2020\)](#).

In both metrics, the measured performance on Leonardo is noticeably slower than the original results for equivalent CPU configurations. Specifically, for the sea ice model CICE running on 24 CPUs, the runtime per step is approximately 0.43 seconds, yielding a CPU-hours-per-step value of 0.0028 hr — significantly higher than the 0.002 hr reported for the same configuration in the reference dataset. The MOM ocean model also demonstrated values above the benchmarked values in [Kiss et al. \(2020\)](#).

This performance gap is primarily attributed to the use of a Spack-managed software stack during porting. While Spack ensures portability and modularity, non system-wide OpenMPI and compiler configurations are not optimized for the Leonardo architecture. In contrast, the original ACCESS-OM2 results were obtained using

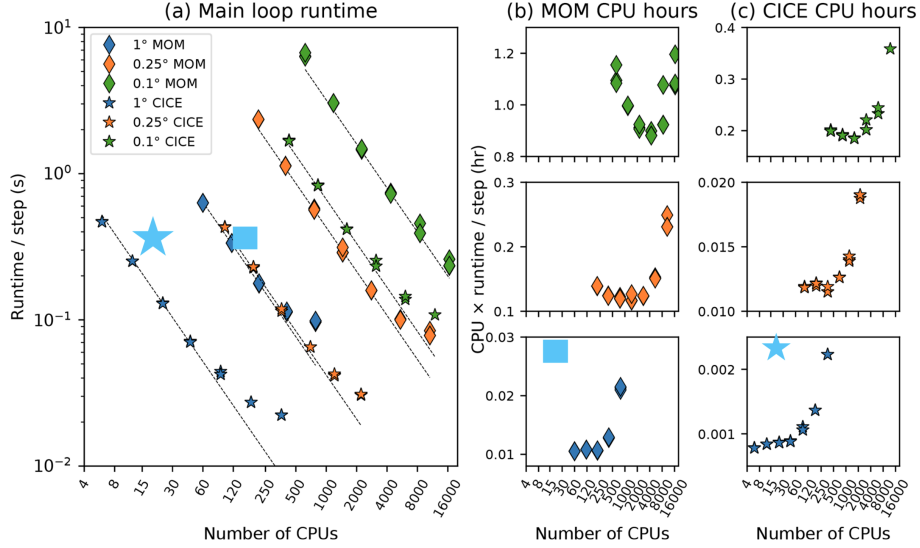


Figure 4.1: ACCESS OM2 runtime estimates in this study (highlighted in cyan) on the top of benchmark runs on Rajin supercomputer [Kiss et al. \(2020\)](#).

site-specific MPI builds and Intel compilers, which are typically fine-tuned to the host system's interconnect topology, memory hierarchy, and CPU instruction set. Future work should involve recompiling ACCESS-OM2 in developer mode using Leonardo's system-wide Intel toolchain and OpenMPI libraries to reduce runtime and improve scaling efficiency.

Figures 4.2 and 4.3 present the default runtime diagnostics produced by the internal timing infrastructure of the CICE (sea ice) and MOM (ocean) model components. These results are obtained from 6-month benchmark experiments using the 1° ACCESS-OM2 configuration.

CICE timings are demonstrated in Figure 4.2, the `TimeLoop` entry corresponds to the total wall time of the model's execution loop. The most significant contributors to runtime are `Diags`, `from_ocn` and `waiting_o`, `ReadWrite` and other non computational processes of the model execution, such as making log files (`Diags`), waiting for the MOM to complete the step, and other "data exchange" related processes. These refer to the time spent receiving data from the ocean and atmosphere components or idling while awaiting incoming coupling fields. This highlights potential bottlenecks related to synchronization delays across model components during coupling exchanges.

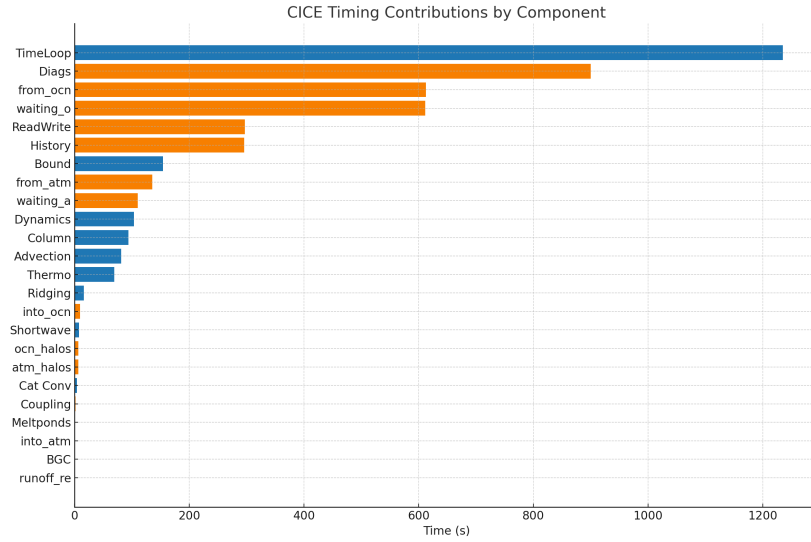


Figure 4.2: CICE5 Runtime contributors, Total Runtime corresponds to the whole ACCESS OM2 experiment, other contribution to the CICE5 processes.

The MOM runtime profile (Fig. 4.3) similarly identifies the `oasis_recv` as a main contributor to the total runtime (Main Loop and Ocean are total experiment and ocean component counters). `oasis_recv` contributor—representing time spent waiting for coupling inputs via the OASIS3-MCT coupler. Other entries such as ocean surface fluxes computation, neutral physics routines, and tracer advection make up smaller but non-negligible fractions of the runtime.

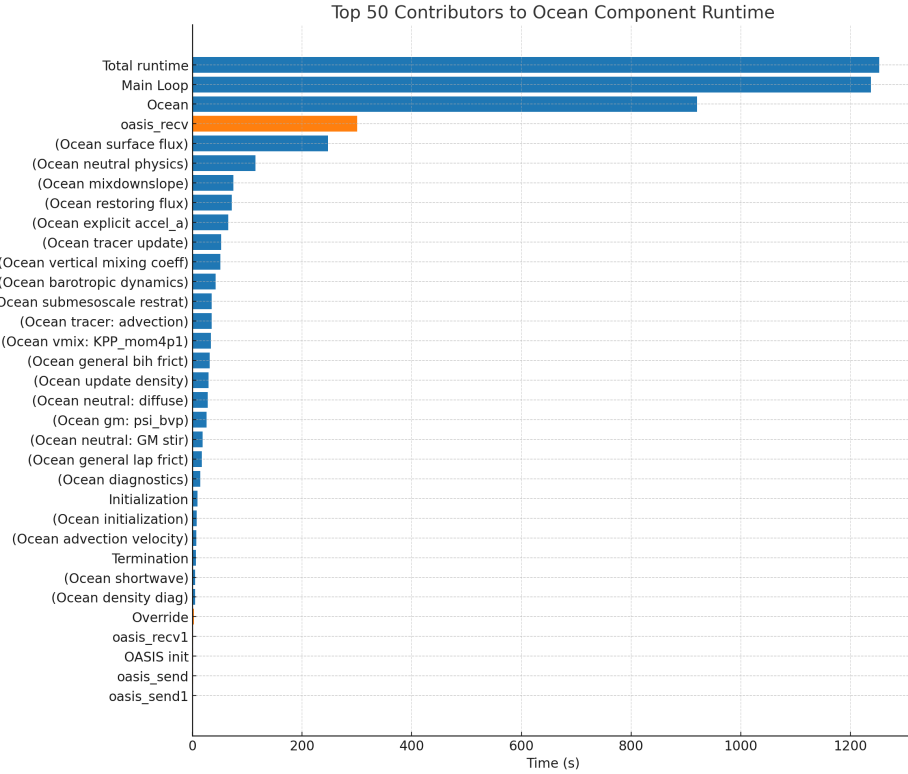


Figure 4.3: ACCESS OM2 Runtime contributors, Total Runtime corresponds to the whole ACCESS OM2 experiment, Ocean - to the MOM5 execution time.

These diagnostics confirm that while physical computations account for a substantial portion of runtime, inter-component communication (particularly via OASIS) and I/O introduce notable overhead. This initial profiling serves as a foundation for more fine-grained performance analysis using tools such as Score-P, PARAVR, or mpiP, and can help guide future optimization strategies.

4.2 Performance Analysis with mpiP: Lightweight MPI Profiling

This chapter provides an in-depth analysis of the MPI performance, focusing on communication patterns and latencies during the coupled model run with mpiP profiling tool. mpiP is a lightweight tool specifically designed for MPI applications. Unlike heavy tracing profilers, mpiP uses statistical sampling to gather function-level information about MPI usage. Operates via LD_PRELOAD, requiring no changes to application source code.

In this study, `mpiP` was used in combination with `perf stat` to collect both MPI-level communication profiles and low-level hardware performance counters for each model component.

Due to the fixed domain decomposition compiled into the model components, only a limited set of processor configurations was compatible with accurate `mpiP` profiling. Specifically, the model components were compiled for the following MPI task counts:

- **MOM5**: 216 MPI ranks, distributed over **2 nodes**.
- **CICE5**: 24 MPI ranks, typically executed on **1 node**.
- **YATM**: a single MPI rank (serial execution).

Consequently, `mpiP` results presented in this chapter are limited to these compatible setup.

MPI Call Frequency

The frequency of MPI calls provides insight into how often different functions were invoked during the simulation. The most frequently called MPI function was `Bcast`, followed by `Recv`, `Barrier` and `Waitall`.

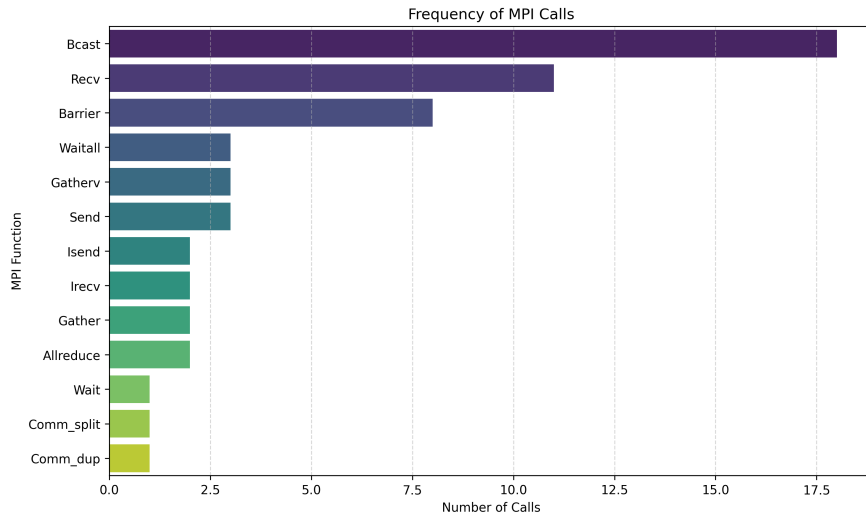


Figure 4.4: MPI Call Frequency.

The `Bcast` function appears as the most frequent MPI call. This is typical in simulations involving frequent updates or data exchange across all processes. `Barrier`

and `Recv` also show up frequently, which is expected as synchronization and receive operations are integral to parallelism in MPI-based models. If synchronization can be reduced, it could lead to improved performance by reducing the frequency of collective calls and barriers.

Files Using MPI

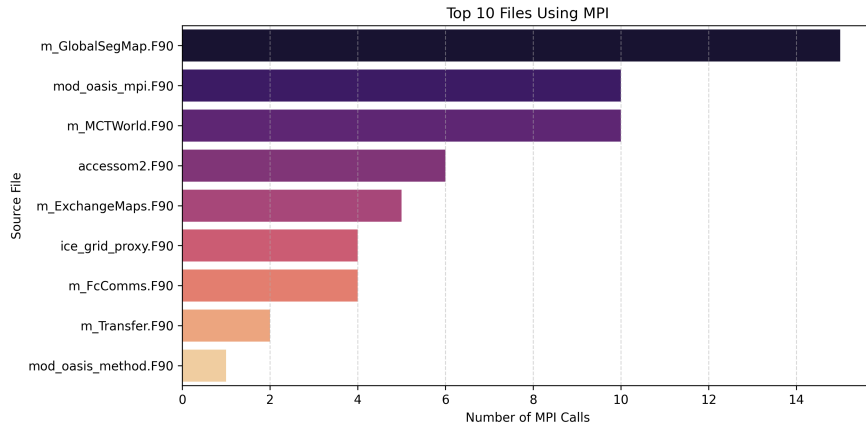


Figure 4.5: Top 10 Files Using MPI.

The file `m_GlobalSegMap.F90` stands out as the file with the highest number of MPI calls. This Fortran code is the OASIS coupler module for the decomposition of an array. `mod_oasis_mpi.F90` and `m_MCTWorld.F90`, and `accessom2.F90` also show significant MPI call activity and also OASIS modules.

MPI Overhead by Rank

The MPI overhead plot (Fig. 4.6) highlights the percentage of time each rank spent in MPI communication relative to the total runtime. When analyzing MPI Overhead by Rank, the key goal is to minimize overhead — that is each MPI rank (process) to spend as much of its time doing useful work (computation) and as little time as possible waiting on communication, synchronization, or idle time. Figure 4.6 presents a breakdown of computational overhead for individual model components—MOM5, CICE5, and YATM—as a function of computational node ID in the ACCESS-OM2 simulation.

MOM5 maintains (70-75%) overhead values across all ranks. CICE5, the sea ice model, exhibits significantly higher overhead (80-90%). This behavior suggests suboptimal scaling in its communication or I/O routines, which may be exacerbated by the use of a Spack-built software stack rather than site-optimized libraries. The I/O-heavy structure and frequent exchange with MOM5 could be contributing factors.

YATM, the file-based atmospheric forcing driver, demonstrater 95% overhead. Since YATM handles prescribed data input rather than solving complex PDEs, its computational cost is negligible and unaffected by domain size or node count.

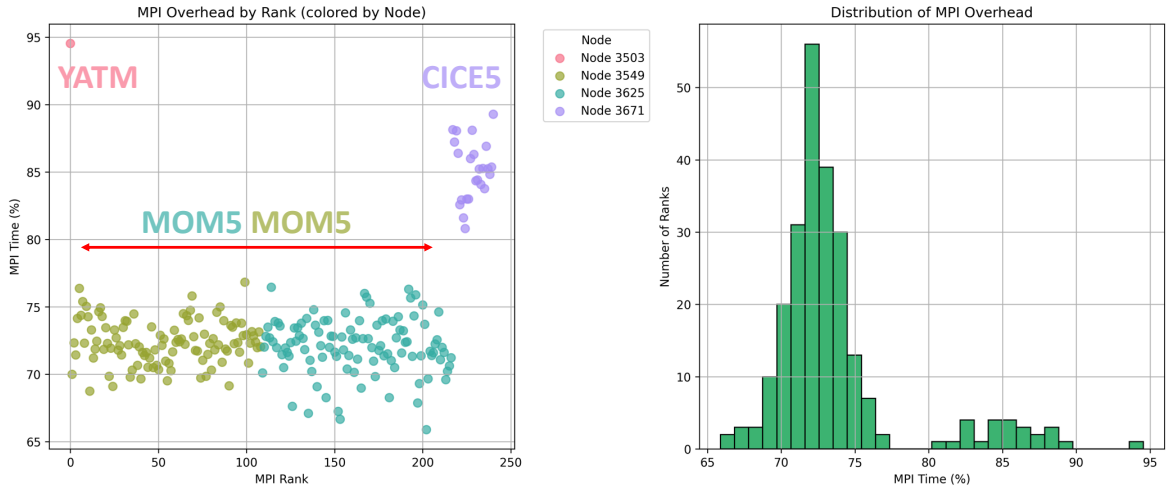


Figure 4.6: MPI Overhead by Rank (colored by Node).

Top MPI Calls by Total Time

Figure 4.7 shows the breakdown of total execution time spent in various MPI routines, based on profiling output collected during a 6-month ACCESS-OM2 simulation. The majority of time is consumed by collective communication calls, particularly `MPI_Bcast` (3647.6 seconds) and `MPI_Allreduce` (1961.8 seconds), which are common in climate models for domain-wide synchronization and global reductions. The presence of `MPI_Waitall` (1791.0 seconds) as the third most time-consuming call indicates significant idle time waiting for message completions, likely due to load imbalance or non-overlapped communication. Point-to-point operations (`MPI_Isend`, `MPI_Recv`) contribute moderately, suggesting a mix of fine-grained and bulk data transfers. Lesser contributions from `MPI_Barrier` and `MPI_Gatherv` indicate

occasional global synchronization and data collection events. These results emphasize that performance tuning efforts should prioritize reducing collective call frequency, overlapping communication with computation, and improving domain decomposition to reduce idle wait times.

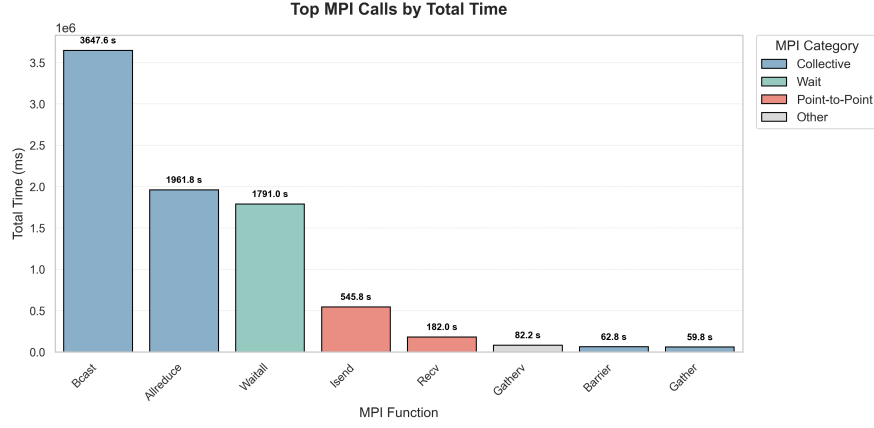


Figure 4.7: Top MPI Calls by Total Time.

Total MPI Sent Data by Callsite

Figure 4.8 shows the top MPI callsites ranked by the total volume of data sent. All entries correspond to non-blocking send operations (`MPI_Isend`), reflecting a communication pattern dominated by asynchronous point-to-point transfers. One callsite, labeled "Isend (Site 25)" stands out significantly with 3767 MB of sent data—more than double the amount sent by other top sites, which are all clustered at 1592.6 MB. This likely corresponds to a central YATM communication routine within the model coupling infrastructure (e.g., ocean-atmosphere or ocean-ice field exchange). The uniformity of sent data across the other callsites suggests regular and balanced communication behavior across distributed subdomains, which is typical for structured grids and conservative remapping schemes used in climate modeling.

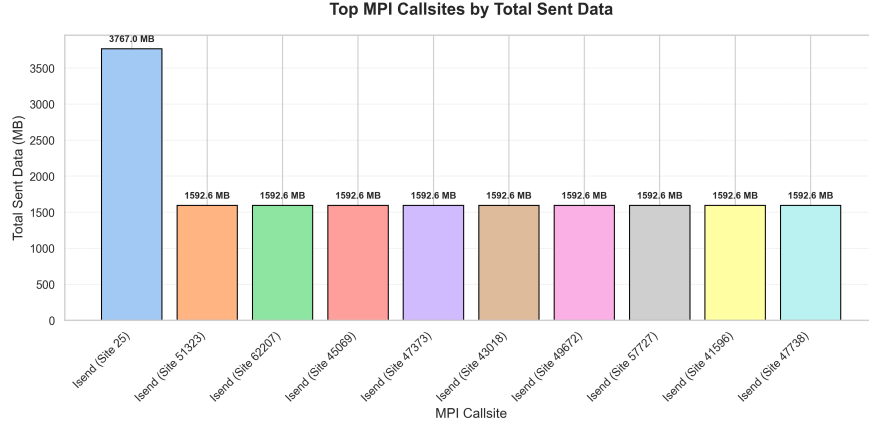


Figure 4.8: Top MPI Callsites by Total Sent Data.

MPI Latency Map (Sender $\rightarrow$ Receiver)

Figure 4.9 presents a synthetic MPI latency map, where each cell represents the estimated average communication latency (in seconds) between a sender and a receiver MPI rank during the ACCESS-OM2 simulation. This estimation is based on profiling output from mpiP, where latency is inferred as the average time difference between matching `MPI_Isend` and `MPI_Recv` events across rank pairs. While this is a coarse approximation—true point-to-point latency measurement would require instrumentation at the communication layer — it provides a qualitative view of communication asymmetries and hotspots.

The plot reveals a non-uniform communication landscape, with certain sender ranks (e.g., 180–200) exhibiting significantly higher average latency toward most peers. These high-latency paths may correspond to inter-node communication bottlenecks, NUMA domain crossings, or load imbalance in coupled model components. The horizontal striping suggests that some sender ranks consistently experience elevated communication delays, which might indicate poor MPI rank placement relative to memory locality or network topology. These insights could guide future NUMA-aware or topology-aware process mapping strategies, especially for large-scale runs.

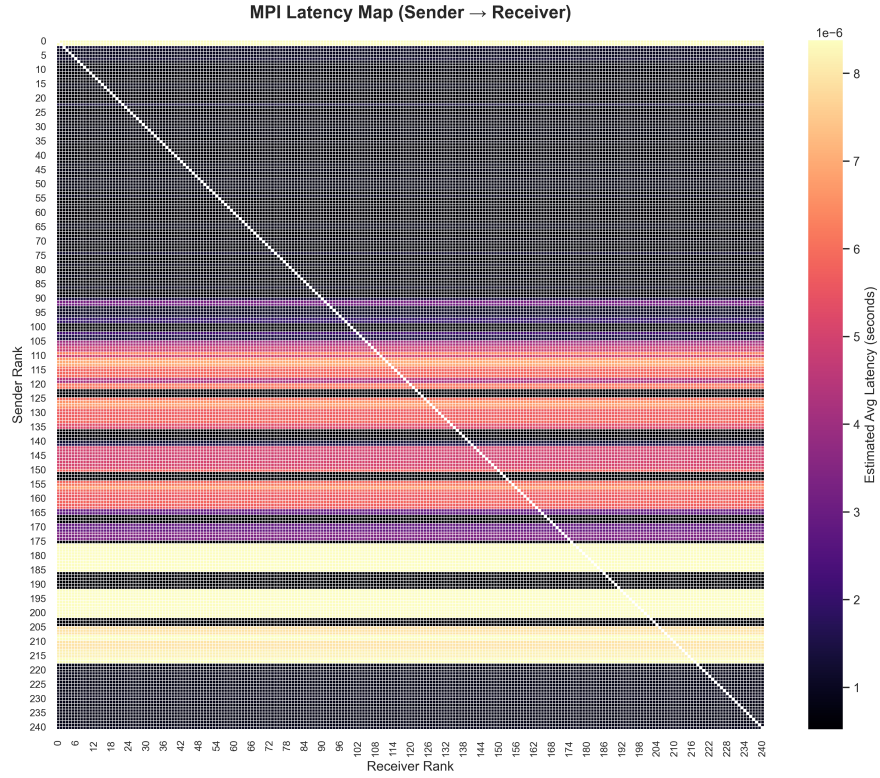


Figure 4.9: MPI Latency Map (Sender → Receiver).

4.3 Profiling with perf tool

The following listing presents raw output from the `perf stat` command used to profile three ACCESS-OM2 executables: `cice`, `fms_ACCESS-OM.x`, and `yatm.exe`. This snapshot summarizes low-level hardware counters, such as instruction throughput (IPC), cache and branch misses, context switches, and page faults.

```
Performance counter stats for cice_aucom_360x300_24x1_24p.exe:

1,468,836,374,484    cycles
4,453,505,239,148    instructions          #    3.03  insn per cycle
 275,233,925        cache-misses
 657,273,923        branch-misses
   22,825           context-switches
    1,047           cpu-migrations
   24,459           page-faults

499.972331149 seconds time elapsed

 60.014479000 seconds user
431.064935000 seconds sys

Performance counter stats for fms_ACCESS-OM.x:
```

```

1,387,453,775,130    cycles
3,887,648,549,459    instructions          #    2.80  insn per cycle
 2,641,249,954      cache-misses
 520,022,733        branch-misses
   13,456           context-switches
    1,036           cpu-migrations
    70,442          page-faults

499.884731680 seconds time elapsed

 93.517250000 seconds user
400.871192000 seconds sys

Performance counter stats for yatm.exe:

1,487,068,444,806    cycles
3,475,577,791,552    instructions          #    2.34  insn per cycle
 4,232,210,569      cache-misses
 1,555,433,355      branch-misses
   41,044           context-switches
    0               cpu-migrations
   468,094          page-faults

501.452621791 seconds time elapsed

183.475472000 seconds user
310.693474000 seconds sys

```

Instructions Per Cycle (IPC)

Instructions Per Cycle (IPC) is a widely used metric for assessing CPU efficiency. It represents the number of instructions a processor completes during each CPU cycle. Higher IPC values indicate greater instruction throughput and typically correlate with more efficient execution.

The IPC metric is computed as:

$$\text{IPC} = \frac{\text{Instructions}}{\text{CPU Cycles}}$$

The profiling results shown in Figure 4.12 focus on test setups rather than production configurations, with all components (and CICE5) compiled for a fixed 24-core layout provided by ACCESS-NRI. The 1-node, 24-core CICE5 configuration matches this layout exactly and yields the highest IPC (approaching 2.9) and lowest time-per-core, representing a near-ideal profiling baseline. As core counts increase (56, 112, and 162), IPC remains high (2.5–2.9), but time-per-core rises due to mismatched

decompositions, idle ranks, and communication overheads.

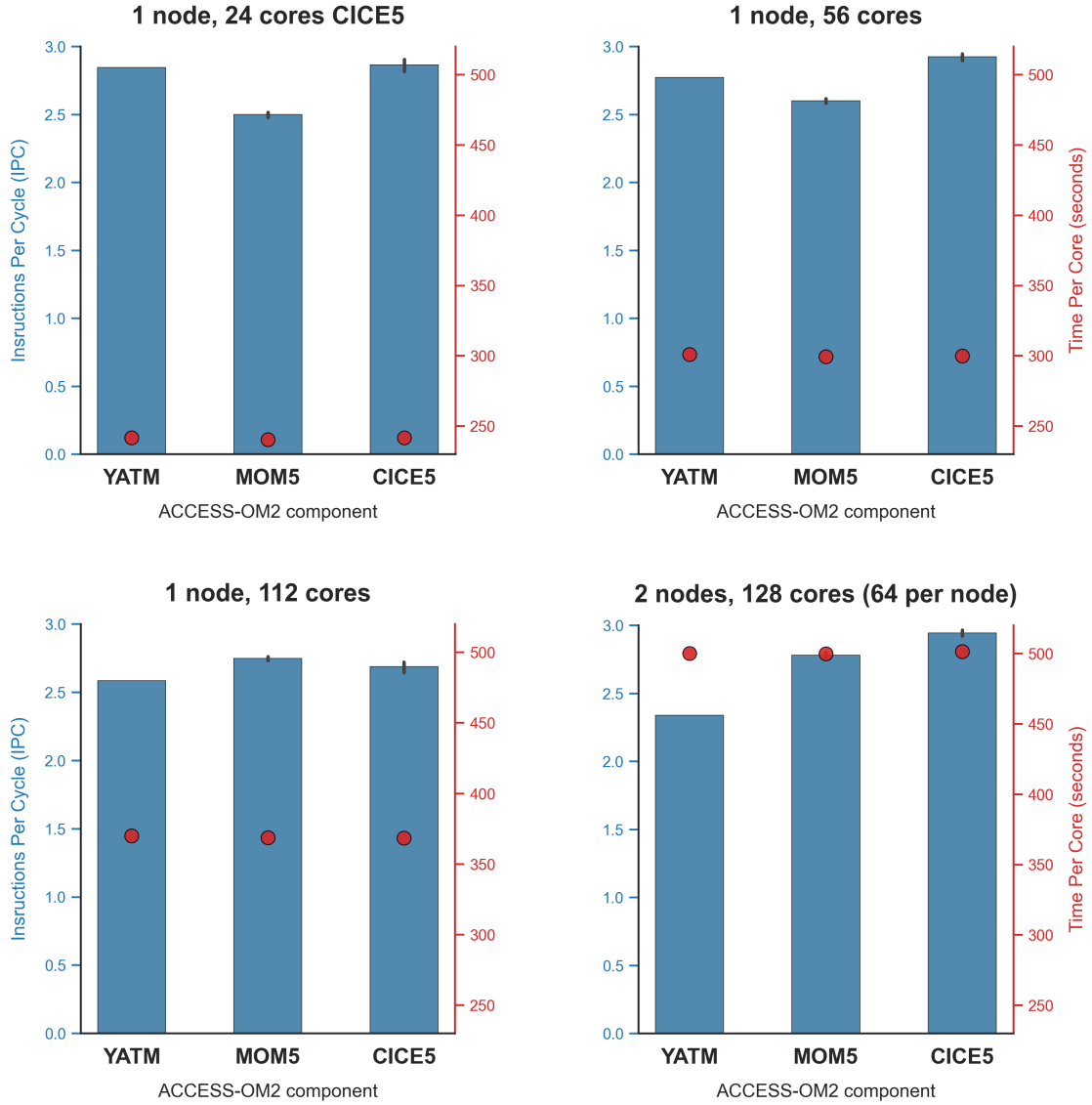


Figure 4.10: Instructions Per Cycle (IPC) across different configurations.

User Time vs. System Time

In performance profiling, two essential time-based metrics help evaluate how compute resources are utilized during program execution:

- **User Time** refers to the amount of time the CPU spends executing user-level code—i.e., the actual scientific computations performed by the model (such as numerical solvers and physics routines).
- **System Time** represents the time spent in the kernel executing system-level

operations such as MPI communication, I/O, memory allocation, and process/thread coordination.

A balanced and well-optimized scientific application typically shows higher user time relative to system time, indicating efficient use of CPU cycles for meaningful computation rather than coordination or waiting.

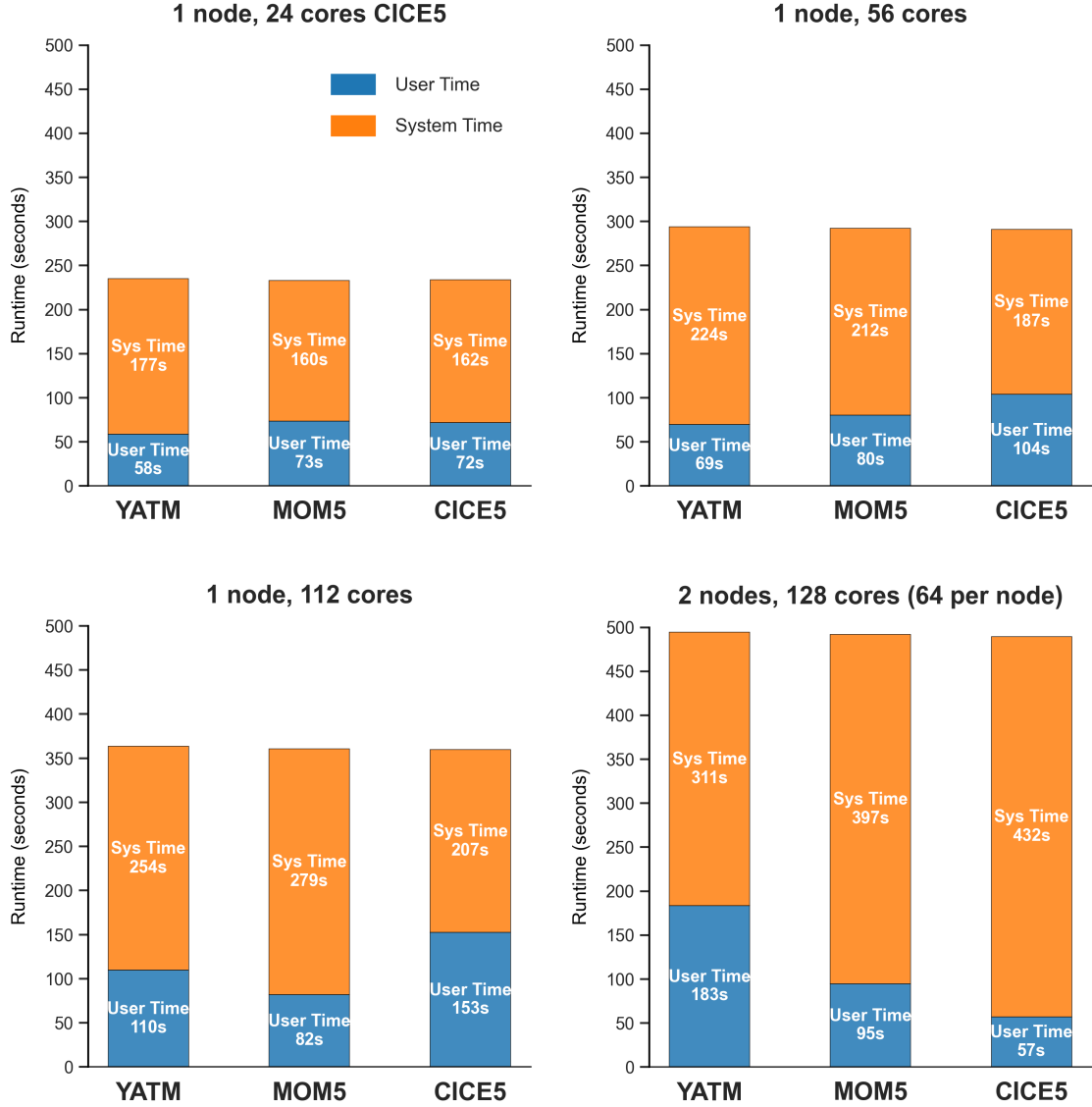


Figure 4.11: System Time and Run Time.

In the case where the system time (~ 160 s) significantly exceeds the user time (~ 73 s), it indicates that a large portion of total runtime is being spent in system-level operations rather than executing application code. This imbalance is usually caused by inefficient resource usage — such as idle CPUs waiting on MPI synchronization, I/O

bottlenecks, or misaligned domain decomposition. In this configuration, the model was executed with 24 CPUs, which matches the CICE compile-time setup. However, the high system time suggests that even though the processor count is correct, other factors — such as I/O contention or suboptimal process placement — may be introducing unnecessary overhead. Ideally, for a well-optimized HPC workload, user time should dominate, with system time remaining comparatively low.

This 24-CPU configuration serves as a reference point, illustrating the importance of aligning execution resources with the compiled domain decomposition. The profiling results validate that CICE5, when executed in its native processor layout, achieves optimal utilization and serves as a baseline for interpreting the performance of other (non-native) configurations. The observed increase in System Time across different configurations, especially when scaling beyond the compiled processor layout, highlights inefficiencies caused by fixed domain decomposition and static resource allocation in the ACCESS-OM2 model. When System Time begins to dominate over User Time, it is a clear indication of performance degradation due to inefficient use of available compute resources. For ACCESS-OM2, this trend confirms the importance of recompiling the model with an appropriate domain decomposition for each target architecture and processor count. Doing so would minimize unnecessary system-level activity and better harness the compute power of modern HPC systems like Leonardo.

Hardware Performance Overheads - perf

A branch predictor is a circuit in modern processors that guesses the outcome of conditional jumps (e.g., if-then-else) to maintain efficient instruction flow in the pipeline. Without it, the CPU must wait until a branch is resolved, stalling execution. By predicting whether a branch will be taken or not, the processor speculatively executes instructions to avoid delays. If the prediction is wrong, the pipeline must discard and restart, costing time—typically 10–20 cycles in long pipelines. To reduce misprediction penalties, branch predictors use historical data from previous jumps to make more accurate guesses over time.

Figure 4.12 present low-level CPU event metrics collected via `perf stat`, normalized per billion instructions and plotted on a logarithmic scale. The figures indicate that **YATM** consistently experiences the highest number of *page faults* and *CPU migrations* across all tested configurations. This could suggest suboptimal memory locality or thread placement for the atmospheric driver on Leonardo.

For **MOM5**, while cache miss and branch miss rates remain stable, page faults are lower compared to YATM, suggesting better memory handling. However, high cache misses—on the order of hundreds of thousands per billion instructions—may reflect the large structured memory layout of the ocean model’s 3D grid.

CICE5 shows relatively balanced behavior. Notably, in the native configuration (24 CPUs), both cache and branch misses are within expected ranges, and page faults and CPU migrations are minimized. This confirms that CICE’s profiling results in the 24-core configuration are the most trustworthy and indicative of actual execution characteristics.

Ideally, well-optimized workloads should have:

- **Low cache and branch misses** (implying good data and control locality),
- **Few page faults** (especially major ones),
- **Minimal CPU migrations** (to preserve cache coherence).

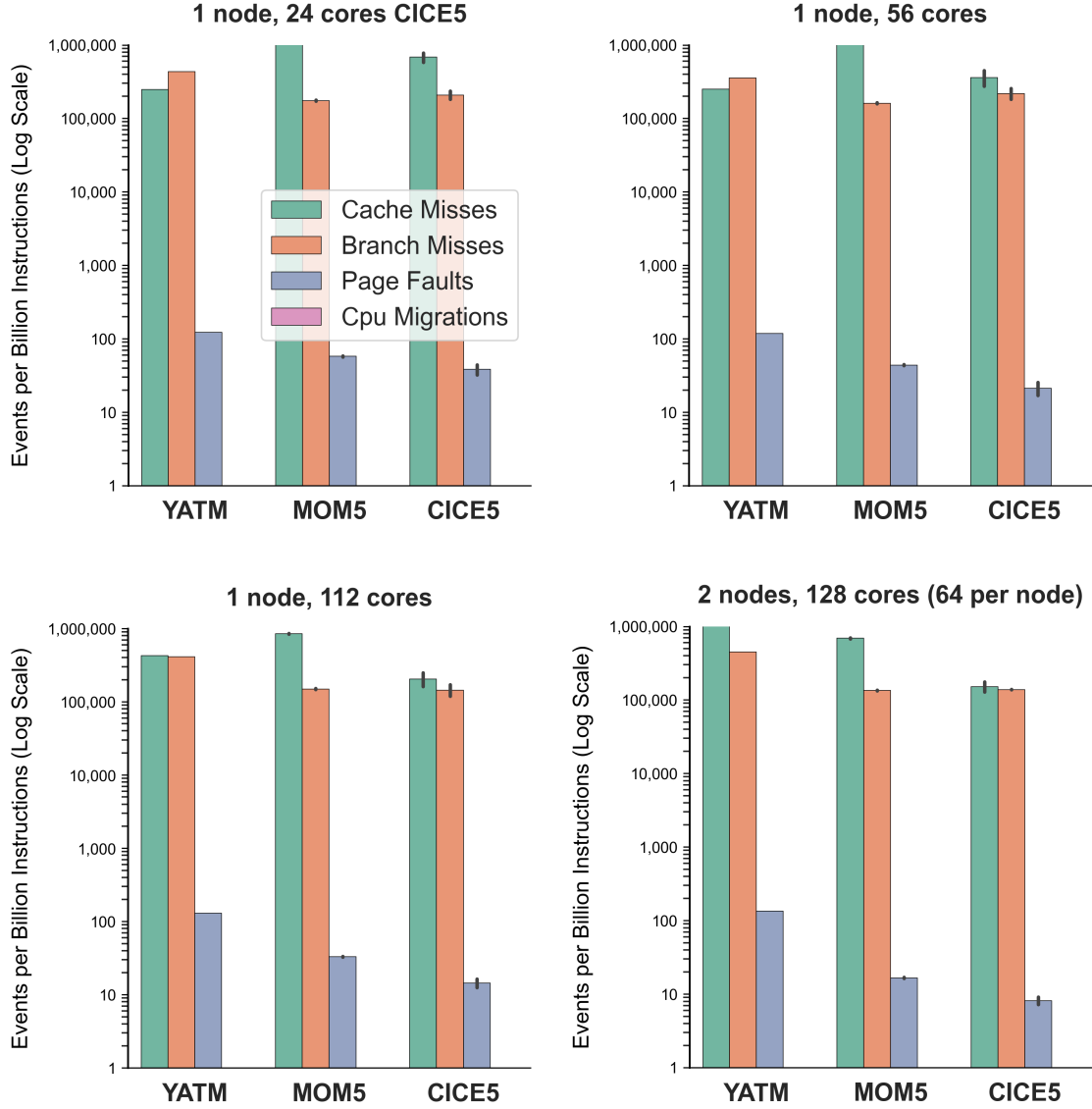


Figure 4.12: Overhead by components.

ACCESS-OM2 was compiled with a fixed processor layout, any execution using more CPUs than required by its internal decomposition results in idle or underutilized resources, artificially inflating wall-clock time and distorting scaling metrics. While profiling experiments varied the number of compute nodes and CPU allocations, the underlying workload distribution remained constant, meaning that key metrics such as instructions per cycle (IPC), system and user time, and OS-level counters like cache misses and context switches should be interpreted as exploratory diagnostics rather than optimized performance indicators. However, these tests still offer valuable insights into how the model behaves at runtime on the Leonardo system. Notably, the

one configuration where CICE5 was run on a single node using 24 cores—matching its domain decomposition at compile time—yielded metrics that accurately reflect real performance, free from the noise of idle processes or mismatched CPU counts. This setup fully utilized available cores and produced reliable IPC and cache behavior data, making it a strong reference point for interpreting sea ice model efficiency. Moving forward, true scaling and performance benchmarking will require recompiling ACCESS-OM2 with layout-specific decompositions and applying compiler or runtime optimizations tailored for Leonardo’s architecture. Doing so would necessitate either new initial condition datasets aligned with each decomposition or automated extensions to the model’s configuration generation process—goals that lie beyond the scope of this study but represent critical next steps for future development.

Statistics for CICE5

Stat	Time Elapsed (sec.)	User Time	Sys Time	Cycles	Instructions	IPC	Cache Misses	Branch Misses	Ctx Switches	CPU Mig.	Page Faults	Cycles /sec
mean	500.03	56.95	432.29	1.45×10^{12}	4.26×10^{12}	2.94	6.36×10^8	5.90×10^8	26 120	1053	34 565	2.89×10^9
median	500.00	55.31	434.42	1.45×10^{12}	4.32×10^{12}	3.00	3.29×10^8	6.00×10^8	25 933	1041	23 802	2.91×10^9
std	0.07	6.88	9.49	2.66×10^{10}	2.09×10^{11}	0.13	6.44×10^8	5.08×10^7	3127	90	25 850	5.33×10^7
min	499.88	42.34	352.28	1.24×10^{12}	3.44×10^{12}	2.40	2.68×10^8	4.47×10^8	13 456	817	22 956	2.48×10^9
max	500.15	93.52	448.31	1.48×10^{12}	4.49×10^{12}	3.07	2.64×10^9	7.57×10^8	52 673	1302	105 489	2.95×10^9
count	147	147	147	147	147	147	147	147	147	147	147	147

Statistics for MOM5

Stat	Time Elapsed (sec.)	User Time	Sys Time	Cycles	Instructions	IPC	Cache Misses	Branch Misses	Ctx Switches	CPU Mig.	Page Faults	Cycles /sec
mean	499.89	94.69	396.87	1.41×10^{12}	3.91×10^{12}	2.78	2.68×10^9	5.24×10^8	12 651	1000	65 051	2.81×10^9
median	499.90	94.79	397.75	1.41×10^{12}	3.96×10^{12}	2.82	2.66×10^9	5.28×10^8	12 234	1025	63 168	2.82×10^9
std	0.10	3.57	5.58	1.52×10^{10}	1.89×10^{11}	0.13	1.54×10^8	4.59×10^7	4073	137	6233	3.06×10^7
min	499.63	84.62	379.21	1.36×10^{12}	3.07×10^{12}	2.17	2.36×10^9	3.81×10^8	6906	647	61 092	2.73×10^9
max	500.05	102.64	408.60	1.42×10^{12}	4.15×10^{12}	2.92	3.17×10^9	6.63×10^8	27 604	1298	93 314	2.85×10^9
count	179	179	179	179	179	179	179	179	179	179	179	179

Statistics for YATM

Stat	Time Elapsed (sec.)	User Time	Sys Time	Cycles	Instructions	IPC	Cache Misses	Branch Misses	Ctx Switches	CPU Mig.	Page Faults	Cycles /sec
summary	501.45	183.48	310.69	1.49×10^{12}	3.48×10^{12}	2.34	4.23×10^9	1.56×10^9	41 044	0	468 094	2.97×10^9
count	1	1	1	1	1	1	1	1	1	1	1	1

5 Summary and Perspectives

5.1 Summary

This thesis presented a comprehensive case study of porting the ACCESS-OM2 Earth System Model from the Gadi supercomputer (Australia) to the Leonardo supercomputer (Italy) — two state-of-the-art HPC systems with distinct architectures, software stacks, and job scheduling environments. The project addressed the increasing need for portability, reproducibility, and performance optimization of complex climate models in the exascale computing era.

ACCESS-OM2 built on MOM5 ocean and CICE5 sea ice models, coupled via OASIS3-MCT and driven by atmospheric reanalyses through the YATM component.

The porting process from Gadi to Leonardo required several major adaptations. These included refactoring Intel-specific build flags, integrating with the Spack-based software stack supported by ACCESS-NRI, recompiling MPI and NetCDF libraries, and resolving inter-node communication issues related to InfiniBand configuration. The migration from PBS to SLURM also required updates to job submission logic and modifications to the `payu` workflow manager components responsible for environment module handling and SLURM-native execution using `srun`. Additionally, runtime profiling tools such as `perf` and `mpiP` were integrated into `payu` for low-overhead performance analysis during coupled model runs.

Simulation output was validated by comparing key diagnostics — such as meridional heat transport and sea ice area — to reference results from ACCESS-OM2 control runs. These results confirmed numerical consistency and reproducibility, demonstrating that the ported model performed as expected on Leonardo’s architecture.

Across all profiling tests applied to ACCESS OM2 runs, the results revealed a strong dominance of MPI communication overhead, with `MPI_Bcast`, `MPI_Allreduce`, and `MPI_Waitall` consuming the majority of total runtime. The coupler (OASIS3-MCT) and YATM emerged as the main bottleneck, with MPI calls within OASIS modules

such as `m_GlobalSegMap.F90` and `mod_oasis_mpi.F90` dominating call frequency and data volume. In particular, the `Isend` operations accounted for the largest share of transferred data, indicating an asynchronous, yet communication-heavy coupling regime. The MPI latency map highlighted imbalanced communication paths between ranks, possibly due to poor rank-to-core placement or NUMA effects.

The profiling results demonstrate that in its current form, ACCESS-OM2 on Leonardo is primarily limited by MPI communication costs rather than raw compute performance. To move forward, recompilation against system-optimized Intel toolchains, NUMA-aware process mapping, and coupling redesign (e.g., reducing blocking MPI operations) are essential steps for achieving scalable and energy-efficient runs in production settings.

5.2 Perspectives

Although the current ACCESS-OM2 installation workflow relies on Spack-managed builds to ensure cross-platform portability and strict dependency isolation, this approach introduces two notable limitations: (1) suboptimal runtime performance due to generic compiler and MPI settings, and (2) limited flexibility for iterative development and recompilation of individual model components.

To address these constraints, a promising future direction involves transitioning to a developer-mode installation, in which ACCESS-OM2 is compiled directly against Leonardo’s system-wide Intel compilers and OpenMPI libraries. This setup would align the model with CINECA’s native, hardware-optimized software stack, enabling access to high-performance compiler flags and MPI versions specifically tuned for the underlying architecture.

This configuration would also support systematic benchmarking of various Intel compiler optimization flags—such as `-O3`, `-ipo`, `-xHost`, and `-qopt-zmm-usage=high`—against the conservative defaults used in Spack. This could reveal substantial performance gains, especially for computationally intensive components like MOM5 and CICE.

In addition to lightweight profiling with `perf` and `mpiP`, advanced tools such as `PARAVER` and `Score-P` can be used for in-depth runtime diagnostics. Such fine-grained diagnostics are essential for identifying hidden performance issues, particularly in high-resolution coupled configurations. Due to ACCESS-OM2’s extensive MPI parallelism and high communication frequency between components, physical placement of processes across NUMA domains can have a major impact on performance. In many cases, restricting model components to a single NUMA domain (e.g., staying within a CPU socket) may outperform full-node utilization by minimizing cross-socket latency and improving memory locality.

These improvements form a critical next step in preparing ACCESS-OM2 to run high-resolution, long-timescale simulations on Leonardo.

To optimize ACCESS-OM2 for contemporary and future supercomputing platforms, the model can be restructured to take advantage of hybrid CPU-GPU systems and complex memory hierarchies. This includes porting key computational kernels—such as ocean advection, tracer transport, or sea ice thermodynamics—to GPU-accelerated backends using OpenACC or OpenMP offload directives. Additionally, NUMA-aware memory placement and fine-grained parallelization strategies, as demonstrated in the CESM-HR and ICON-MSA case studies, could significantly enhance performance on CPU-based clusters. Hybrid MPI/OpenMP parallelism can also be implemented consistently across components to reduce communication overhead and improve intra-node efficiency, particularly in tightly coupled subsystems. Coupling exchanges via the OASIS3-MCT interface can be further optimized using asynchronous communication and advanced MPI collectives to reduce latency and load imbalance.

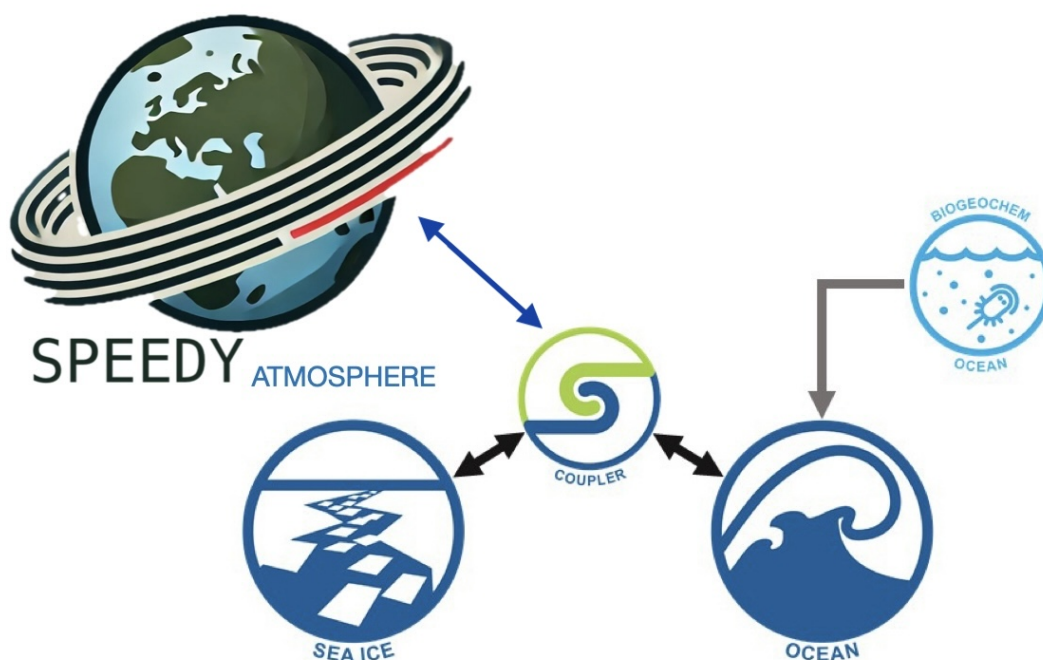
The I/O and coupling overheads—two of the most prominent bottlenecks in CMIP-class simulations—require targeted optimization. On the coupling side, restructuring domain decompositions to minimize idle tiles and applying “sweet spot” analysis can reduce synchronization delays between components. Given that ACCESS-OM2 currently relies on fixed domain layouts that are not dynamically adaptable, integrating auto-tuning tools for processor layout selection (such as ELPiN

or SCORPIO) would allow for more scalable and efficient resource utilization. Together, these improvements will ensure that ACCESS-OM2 remains not only scientifically robust and portable, but also increasingly competitive in terms of computational cost, scalability, and energy efficiency as Earth System Models evolve in the exascale computing era.

Coupling SPEEDY with ACCESS-OM2: Toward a Lightweight Interactive Climate System

In its current configuration, ACCESS-OM2 is forced by a prescribed atmosphere via YATM (Yet Another Atmospheric Model), a lightweight driver that applies surface fluxes from reanalysis products such as JRA55-do. While ideal for controlled experiments and model intercomparison projects, this configuration lacks two-way atmosphere–ocean–ice coupling, limiting the system’s ability to simulate interactive climate dynamics.

A promising future development involves replacing YATM with SPEEDY — ICTP Atmospheric General Circulation Model [Molteni and Kucharski \(2016\)](#). SPEEDY offers a balance between computational affordability and dynamical realism, making it well-suited for coupling with high-resolution ocean–sea ice models such as ACCESS-OM2.



Key advantages of this configuration include:

- Introducing Dynamic Coupling:** Incorporating SPEEDY provides a live atmospheric component, allowing ACCESS-OM2 to simulate interactive feedbacks between the ocean, atmosphere, and sea ice. This includes processes like SST–wind coupling, ice–albedo feedbacks, and spontaneous climate variability (e.g., ENSO, SAM).
- Computational Efficiency Compared to ACCESS-ESM:** In contrast to the full ACCESS-ESM framework, which includes complex biogeochemical and land-surface components and requires significant HPC resources, SPEEDY is lightweight and modular. It enables researchers to explore coupled dynamics without the overhead of a full Earth System Model.
- Millennial-Scale Simulations at Low Cost:** SPEEDY’s efficiency would allow for the coupled system to support *thousand-year simulations*, ideal for paleoclimate studies, equilibrium sensitivity experiments, and assessment of long-term ocean–atmosphere feedbacks — which are infeasible in full ESM

configurations.

- **Ideal for Sensitivity and Process Studies:** The SPEEDY–ACCESS-OM2 setup offers a flexible, tunable platform to explore specific aspects of coupled variability, such as ocean–atmosphere teleconnections, the role of surface flux feedbacks, or changes in the overturning circulation.

In summary, coupling ACCESS-OM2 with SPEEDY would unlock the power of interactive climate modeling at intermediate complexity — enabling a flexible, efficient, and scientifically rich framework for exploring long-term climate dynamics, with simulations spanning **thousands of years**, but at a fraction of the cost of full ESMs.

REFERENCES

- Acosta, M. C., and Coauthors, 2024: The computational and energy cost of simulation and storage for climate science: lessons from cmip6. *Geoscientific Model Development*, **17**, 3081–3098, doi:[10.5194/gmd-17-3081-2024](https://doi.org/10.5194/gmd-17-3081-2024), URL <https://gmd.copernicus.org/articles/17/3081/2024/>.
- Adams, S. V., and Coauthors, 2019: Lfric: Meeting the challenges of scalability and performance portability in weather and climate models. *Journal of Parallel and Distributed Computing*, **132**, 383–396, doi:<https://doi.org/10.1016/j.jpdc.2019.02.007>, URL <https://www.sciencedirect.com/science/article/pii/S0743731518305306>.
- Bishnoi, A., and Coauthors, 2024: Earth system modeling on modular supercomputing architecture: coupled atmosphere–ocean simulations with icon 2.6.6-rc. *Geoscientific Model Development*, **17**, 261–273, doi:[10.5194/gmd-17-261-2024](https://doi.org/10.5194/gmd-17-261-2024), URL <https://gmd.copernicus.org/articles/17/261/2024/>.
- CINECA, 2023: Leonardo supercomputer user guide. Retrieved from <https://www.hp.cineca.it/systems/hardware/leonardo/>.
- Du, S., Y. Zhao, Z. Tian, D. Geisler-Moroder, and W. Wang, 2025: Exploring gpu acceleration framework for climate based daylight modeling. *Building Simulation*, **18**, 33–46, doi:[10.1007/s12273-024-1207-5](https://doi.org/10.1007/s12273-024-1207-5), URL <https://doi.org/10.1007/s12273-024-1207-5>.
- Eyring, V., S. Bony, G. A. Meehl, C. A. Senior, B. Stevens, R. J. Stouffer, and K. E. Taylor, 2016: Overview of the Coupled Model Intercomparison Project Phase 6 (CMIP6) experimental design and organization. *Geoscientific Model Development*, **9** (5), 1937–1958, doi:[10.5194/gmd-9-1937-2016](https://doi.org/10.5194/gmd-9-1937-2016), URL <https://gmd.copernicus.org/articles/9/1937/2016/>.
- Griffies, S. M., and Coauthors, 2016: OMIP contribution to CMIP6: experimental and diagnostic protocol for the physical component of the Ocean Model Intercomparison Project. *Geoscientific Model Development*, **9** (9), 3231–3296, doi:[10.5194/gmd-9-3231-2016](https://doi.org/10.5194/gmd-9-3231-2016), URL <https://gmd.copernicus.org/articles/9/3231/2016/>.
- Gysi, T., C. Osuna, O. Fuhrer, M. Bianco, and T. C. Schulthess, 2015: Stella: A domain-specific tool for structured grid methods in weather and climate

- models. *Proceedings of the international conference for high performance computing, networking, storage and analysis*, 1–12.
- Haarsma, R., and Coauthors, 2020: Highresmip versions of ec-earth: Ec-earth3p and ec-earth3p-hr – description, model computational performance and basic validation. *Geoscientific Model Development*, **13**, 3507–3527, doi:[10.5194/gmd-13-3507-2020](https://doi.org/10.5194/gmd-13-3507-2020), URL <https://gmd.copernicus.org/articles/13/3507/2020/>.
- Kiss, A. E., and Coauthors, 2020: ACCESS-OM2 v1.0: a global ocean–sea ice model at three resolutions. *Geoscientific Model Development*, **13** (2), 401–442, doi:[10.5194/gmd-13-401-2020](https://doi.org/10.5194/gmd-13-401-2020), URL <https://gmd.copernicus.org/articles/13/401/2020/>.
- Leyton, S. K., and T. J. Osborne, 2008: A quantum algorithm to solve nonlinear differential equations. *arXiv preprint arXiv:0812.4423*.
- Lloyd, S., G. De Palma, C. Gokler, B. Kiani, Z.-W. Liu, M. Marvian, F. Tennie, and T. Palmer, 2020: Quantum algorithm for nonlinear differential equations. *arXiv preprint arXiv:2011.06571*.
- Locarnini, R. A., and Coauthors, 2013: World ocean atlas 2013, volume 1: Temperature. *NOAA Atlas NESDIS 73*, available at: <https://www.ncei.noaa.gov/archive/accession/NCEI-WOA13>.
- Molteni, F., and F. Kucharski, 2016: Description of the ictp agcm (speedy). version 40. *Abdus Salam International Centre for Theoretical Physics, Trieste, Italy*.
- National Computational Infrastructure (NCI), 2023: Gadi user guide. Retrieved from <https://opus.nci.org.au/display/Help/Gadi+User+Guide>.
- Nielsen, M. A., and I. L. Chuang, 2010: *Quantum computation and quantum information*. Cambridge university press.
- Palmer, T. N., 2019: Stochastic weather and climate models. *Nature Reviews Physics*, **1** (7), 463–471, doi:[10.1038/s42254-019-0062-2](https://doi.org/10.1038/s42254-019-0062-2), URL <https://doi.org/10.1038/s42254-019-0062-2>.
- Rasch, P. J., and Coauthors, 2019: An overview of the atmospheric component of the energy exascale earth system model. *Journal of Advances in Modeling Earth Systems*, **11**, 2377–2411, doi:<https://doi.org/10.1029/2019MS001629>, URL <https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2019MS001629>.
- Tennie, F., and T. N. Palmer, 2023: Quantum Computers for Weather and Climate Prediction: The Good, the Bad, and the Noisy. *Bulletin of the American*

- Meteorological Society*, **104** (2), E488 – E500, doi:[10.1175/BAMS-D-22-0031.1](https://doi.org/10.1175/BAMS-D-22-0031.1), URL <https://journals.ametsoc.org/view/journals/bams/104/2/BAMS-D-22-0031.1.xml>.
- Tsujino, H., and Coauthors, 2018: Jra-55 based surface dataset for driving ocean-sea-ice models (jra55-do). *Ocean Modelling*, **130**, 79–139, doi:<https://doi.org/10.1016/j.ocemod.2018.07.002>, URL <https://www.sciencedirect.com/science/article/pii/S146350031830235X>.
- Vazhkudai, S. S., and Coauthors, 2018: The design, deployment, and evaluation of the coral pre-exascale systems. *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, 661–672, doi:[10.1109/SC.2018.00055](https://doi.org/10.1109/SC.2018.00055).
- Xu, S., X. Huang, L.-Y. Oey, F. Xu, H. Fu, Y. Zhang, and G. Yang, 2015: Pom.gpu-v1.0: a gpu-based princeton ocean model. *Geoscientific Model Development*, **8**, 2815–2827, doi:[10.5194/gmd-8-2815-2015](https://doi.org/10.5194/gmd-8-2815-2015), URL <https://gmd.copernicus.org/articles/8/2815/2015/>.
- Yang, R., M. Ward, and B. Evans, 2020: Parallel i/o in flexible modelling system (fms) and modular ocean model 5 (mom5). *Geoscientific Model Development*, **13**, 1885–1902, doi:[10.5194/gmd-13-1885-2020](https://doi.org/10.5194/gmd-13-1885-2020), URL <https://gmd.copernicus.org/articles/13/1885/2020/>.
- Ye, Y., and Coauthors, 2022: swnemo_v4.0: an ocean model based on NEMO4 for the new-generation Sunway supercomputer. *Geoscientific Model Development*, **15** (14), 5739–5756, doi:[10.5194/gmd-15-5739-2022](https://doi.org/10.5194/gmd-15-5739-2022), URL <https://gmd.copernicus.org/articles/15/5739/2022/>.
- Yepes-Arbós, X., G. van den Oord, M. C. Acosta, and G. D. Carver, 2022: Evaluation and optimisation of the i/o scalability for the next generation of earth system models: Ifs cy43r3 and xios 2.0 integration as a case study. *Geoscientific Model Development*, **15**, 379–394, doi:[10.5194/gmd-15-379-2022](https://doi.org/10.5194/gmd-15-379-2022), URL <https://gmd.copernicus.org/articles/15/379/2022/>.
- Zhang, S., and Coauthors, 2020: Optimizing high-resolution community earth system model on a heterogeneous many-core supercomputing platform. *Geoscientific Model Development*, **13**, 4809–4829, doi:[10.5194/gmd-13-4809-2020](https://doi.org/10.5194/gmd-13-4809-2020), URL <https://gmd.copernicus.org/articles/13/4809/2020/>.
- Zweng, M. M., and Coauthors, 2013: World ocean atlas 2013, volume 2: Salinity. *NOAA Atlas NESDIS 74*, available at: <https://www.ncei.noaa.gov/archive/accession/NCEI-WOA13>.